

Database Systems (DVLM3,DAT5,SW5) Storage + Indexes

Yumeng Song and Juan Manuel Rodriguez

Department of computer science, Aalborg University

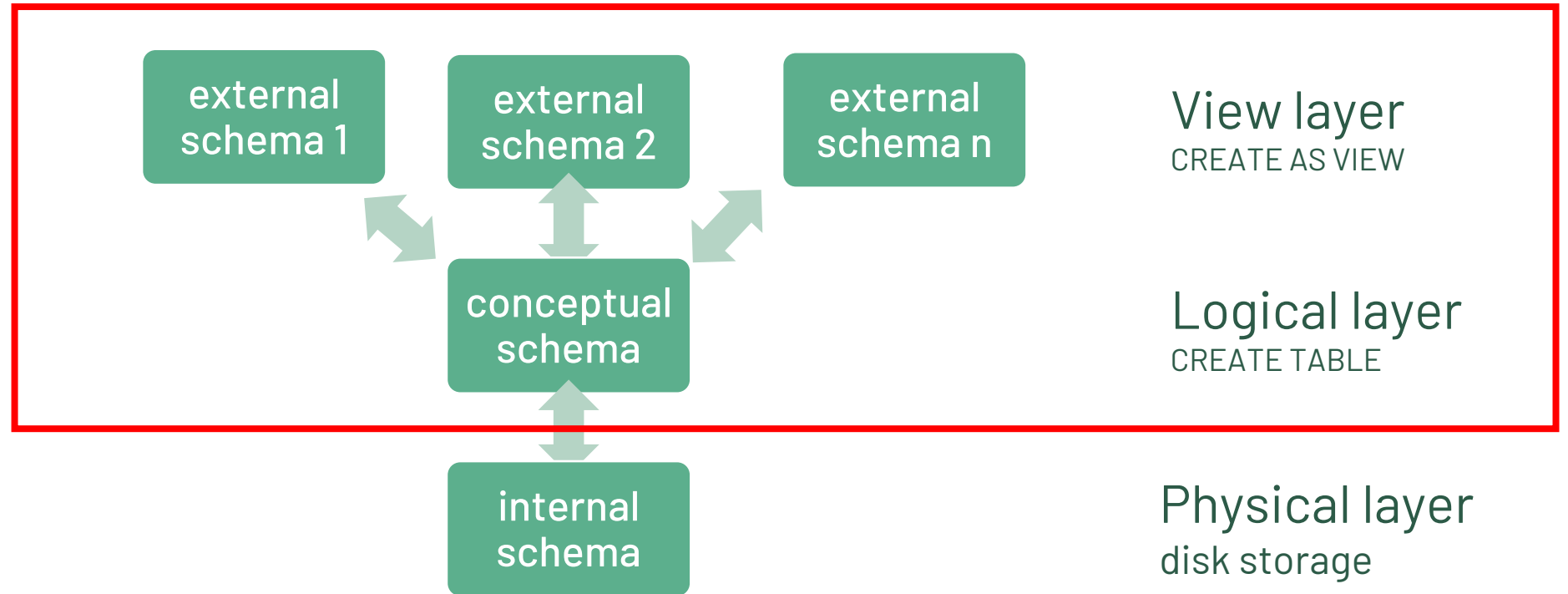
Fall 2025



AALBORG
UNIVERSITY



DBMS Abstraction Layers



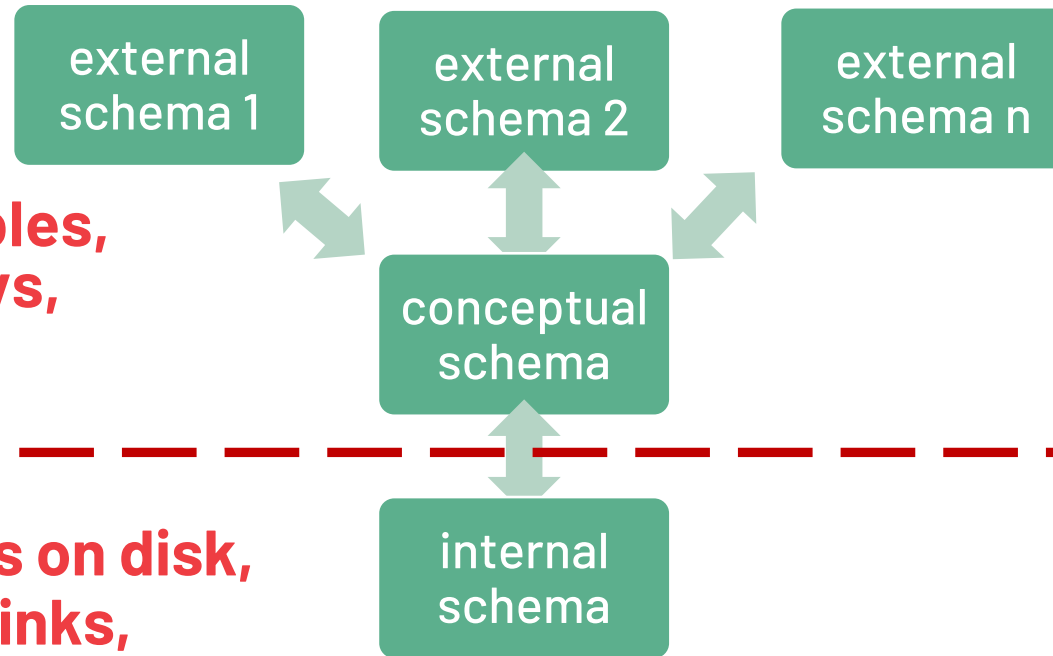
A DBMS ensures:

- **Physical data independence:** changes on the physical layer have no influence on the logical layer



DBMS Abstraction Layers

Here we talk about Tables,
Tuples, Attributes, Keys,
Foreign Keys



View layer
CREATE AS VIEW

Logical layer
CREATE TABLE

Here we talk about files on disk,
bits & bytes, pointers, links,
memory addresses

Physical layer
disk storage

Outline

1. Storage Architecture

- DBMS Stack ★
- Memory hierarchy ★
- Disk Oriented DBMS ★
- Sequential/Random access ★
- Page storage Architecture ★

2. Representation on Disk

- Page layout ★
- Tuple storage + tuple layout ★
- Row storage vs column storage ★
- Denormalized storage

3. Indexing

- Data Structures ★
- Assumption / requirements ★
- Types of indexes: Hash / Tree + Static / Dynamic ★
- Hash Table / Hash Index Variants ★
- Tree index ★
- Primary key and clustering ★
- Sparse / dense ★

4. When to use indexes & effect on queries

- selecting which index to build ★
- partial keys ★
- selecting which index is compatible with your query ★

★ **Core Concepts: you need to understand these**

Based on chapter & online slides:

12.1, 12.3, 12.6, 13.1-13.3, 14.1-14.3, 14.5

- from Database System Concepts 7th edition ; Silberschatz et al.

Intro to Database Systems (15-445/645), Andy Pavlo CMU

- <https://15445.courses.cs.cmu.edu/fall2022/>





DBMS Internals

- ▶ We know how to use a DBMS
- ▶ We will next learn how to build software that manages a database (i.e., a DBMS).
- ▶ We will look into a DISK-BASED architecture: DBMS assumes that the primary storage location of the database is on non-volatile disk.

Query Planning

Operator Execution

Access Methods

Buffer Pool Manager

Disk Manager



Storage Hierarchy

Query Planning

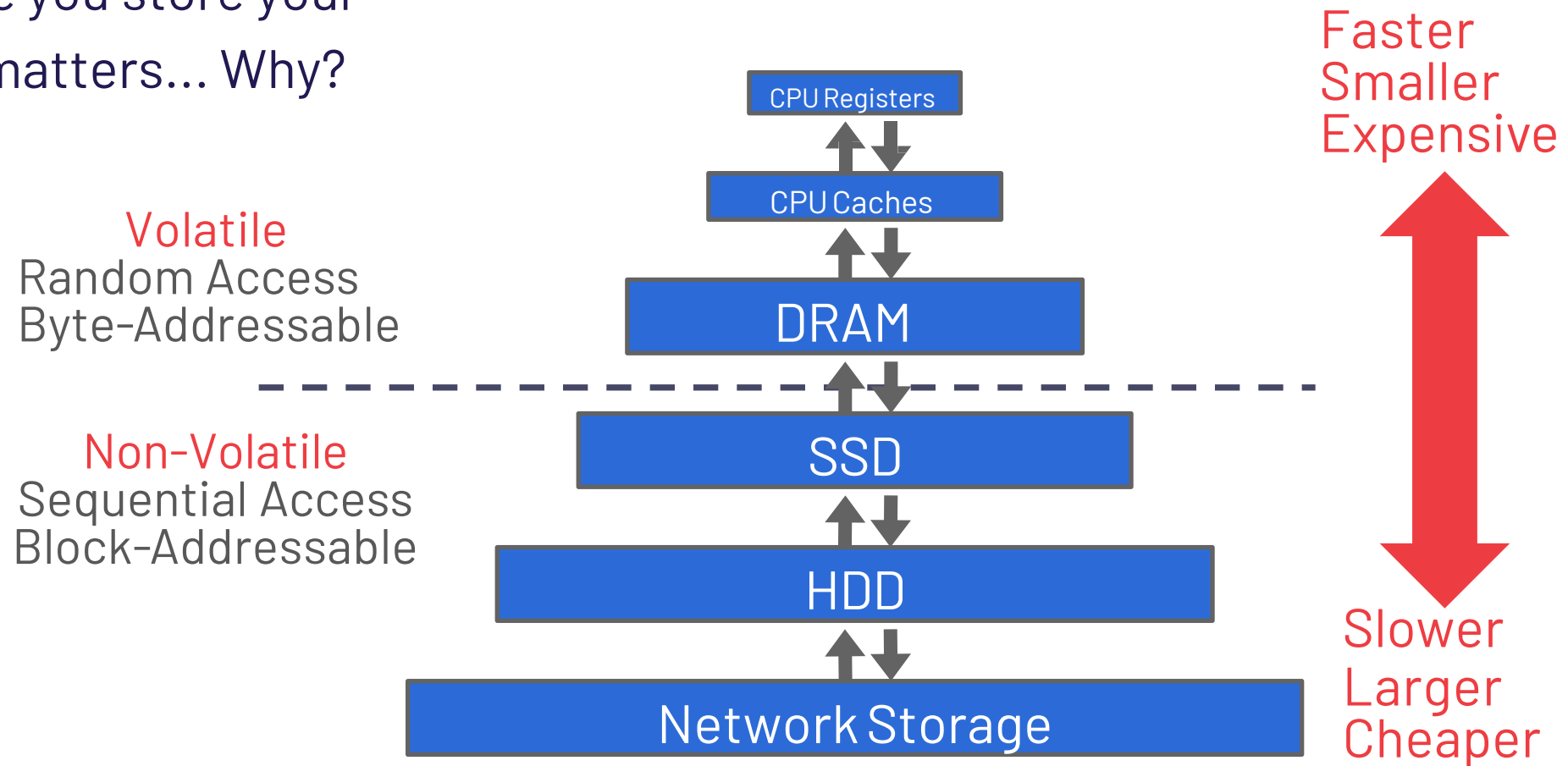
Operator Execution

Access Methods

Buffer Pool Manager

Disk Manager

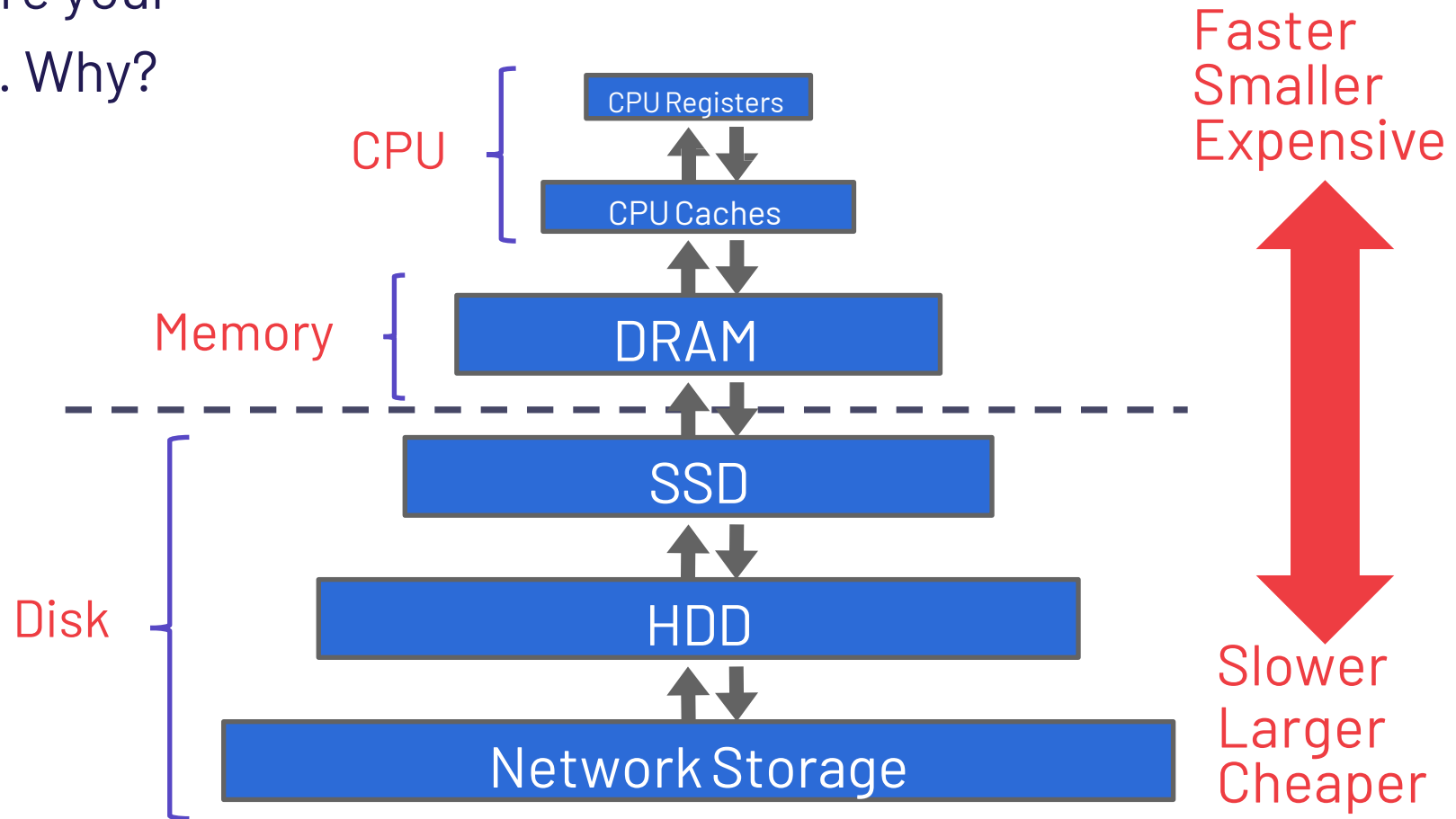
Where you store your data matters... Why?





Storage Hierarchy

Where you store your data matters... Why?



Query Planning

Operator Execution

Access Methods

Buffer Pool Manager

Disk Manager

Storage Hierarchy → Access Time

Latency Numbers			If we pretend to be: 1ns ↦ 1sec
(approximate figures)			
1 ns	L1Cache Ref	512 KB	← 1sec
4 ns	L2 Cache Ref	4 MB	← 4 sec
100 ns	DRAM	16 GB	← 100 sec

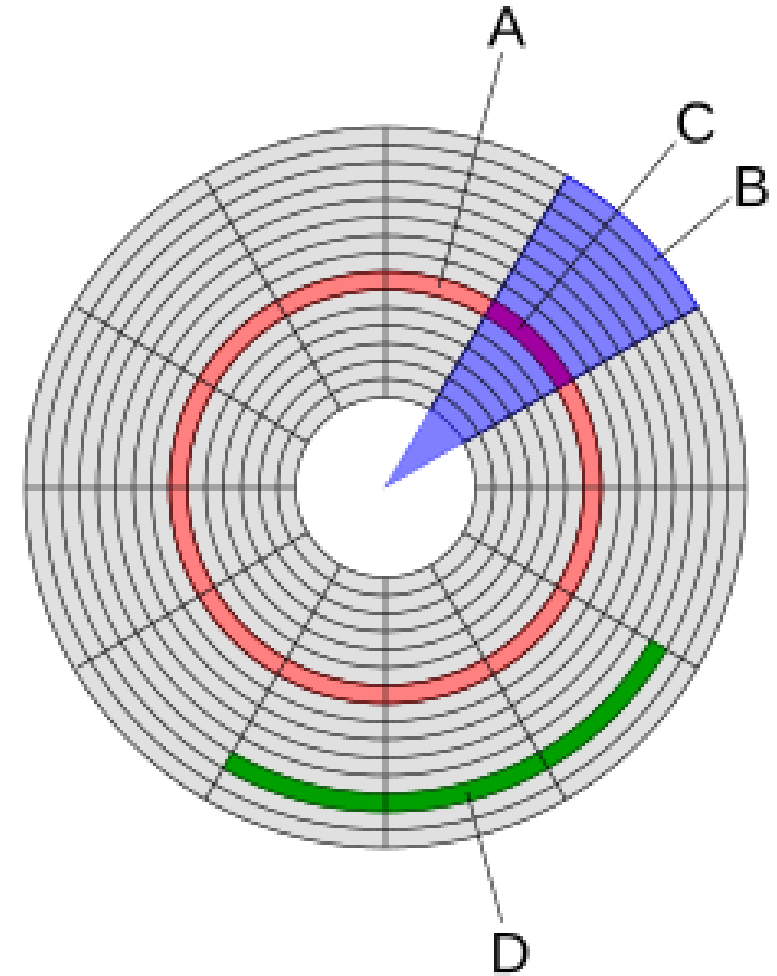
16,000 ns	SSD	1 TB	← 4.4 hours
2,000,000 ns	HDD	+8TB	← 3.3 weeks
~50,000,000 ns	Network Storage		← 1.5 years
1,000,000,000 ns	Tape Archives	+1PB	← 31.7 years

FYI: 1 second = 10^9 ns

[Tape Storage Solutions | IBM](#)

File System/Disk organization

- Disk are structure in sectors.
- OS organized the file systems in blocks. (NTFS and Ext4 uses a default of 4Kb.)
- Matches disk I/O behavior (disks read/write blocks)
- Easier caching, buffering, and allocation.
- Reduces I/O overhead and fragmentation.
- Enables page-level locking and journaling.



[NTFS overview | Microsoft Learn](#)

[2. Data Structures and Algorithms — The Linux Kernel documentation](#)

Physical structure of disks

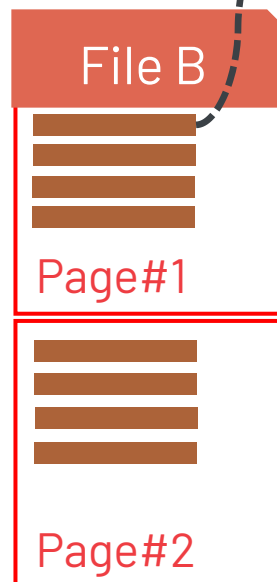
Logical layer

CREATE TABLE ...
SELECT ...

Name	Rank	Grade
Jens	C3	12
Zack	C4	7
Jane	C2	10
July	C6	6
Jill	C8	4
...	...	...

Physical Layer

Files,
Pages,
Records,
Fields



Record

R987	J	o	h	n	_	D	o	e	C	4	1	2
------	---	---	---	---	---	---	---	---	---	---	---	---

Field



File Storage

(Disk based) The DBMS stores a database as one or more files on disk typically in a proprietary format.

→ The OS doesn't know anything about the contents of these files.

- The database is stored as a collection of **files**
- Files are stored as one or more **blocks** (a.k.a. **pages**)
- Each file corresponds to a set of **records**
- A record contains multiple **fields**
- Some block/pages contain **auxiliary data**, meta-data, indexes, log records

We will focus first on Blocks/Pages
Each Page is given an identifier (ID)

The DBMS goal is to **manage databases that exceed the amount of memory (RAM) available.**

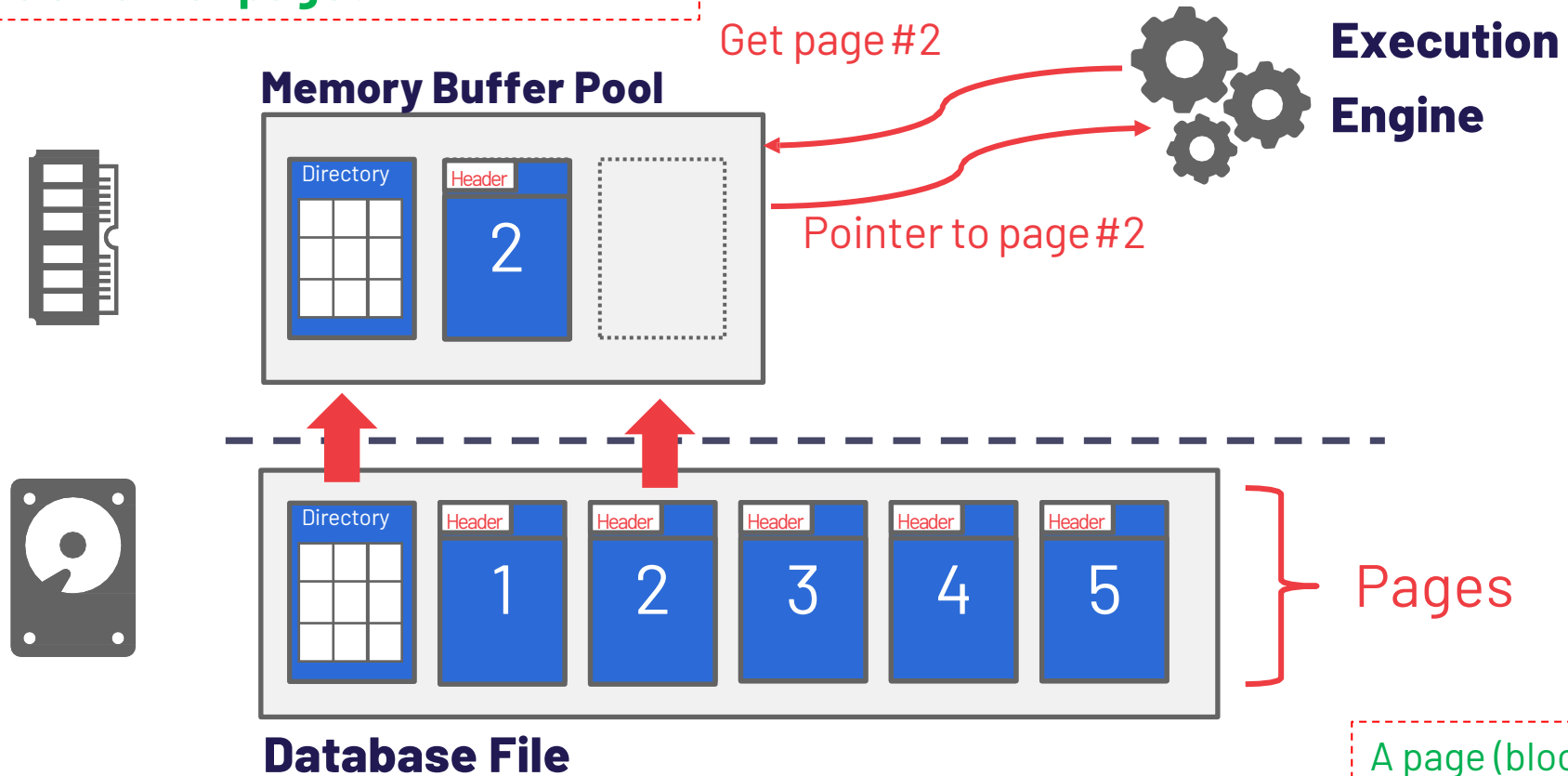
Reading/writing to disk is expensive, it must be managed carefully (localized access!) to **avoid stalls.**

Random access on disk is much slower than sequential access (see later slides): the DBMS will try to maximize the latter



Disk-Oriented DBMS

A directory page is a special page that has addresses to all other pages



A page (block) is the **smallest unit** that can be transferred to/from disk

Database Pages

There are **three different notions of "pages"** in a DBMS:

1. Hardware pages (usually 4KB)
2. OS Pages (usually 4KB)
3. Database Pages (512B – 16KB)

A **hardware page** is the largest block of data that the storage device can **guarantee failsafe writes** to.

The DBMS manages itself how pages are distributed in files!

4KB



ORACLE®

8KB



16KB



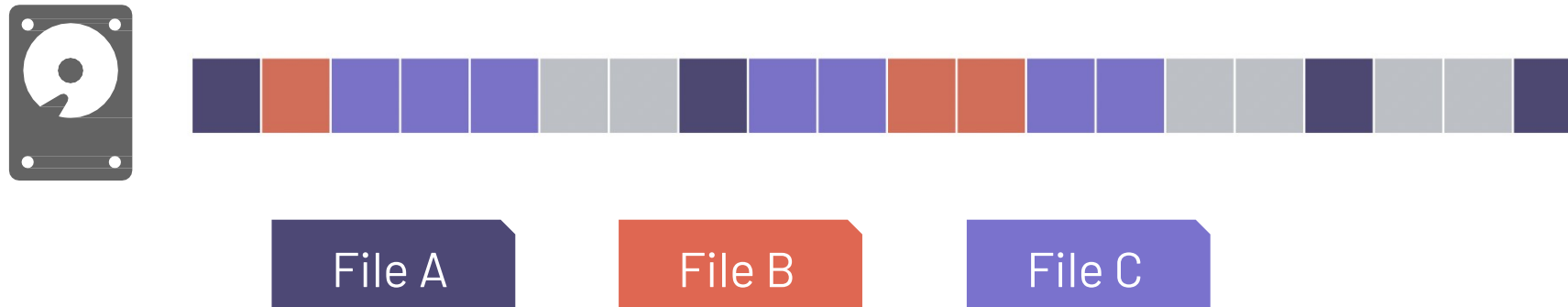
Usually when database pages store data (tuples!)
they store only the tuples of 1 relation (with exceptions, see later)



Sequential vs. Random Access

Random access on non-volatile storage is almost always **much slower** than sequential access. DBMS will want to maximize sequential access.

Files are **not always allocated in contiguous blocks** (pages) on disk!



- Algorithms try to reduce number of writes to random pages so that data is stored in contiguous blocks.
- Allocating multiple pages at the same time is called an extent, it can allow to have more contiguous space
- OS can reorganize file pages on disk, DBMS can reorganize pages across files

Heap File & Page Directory

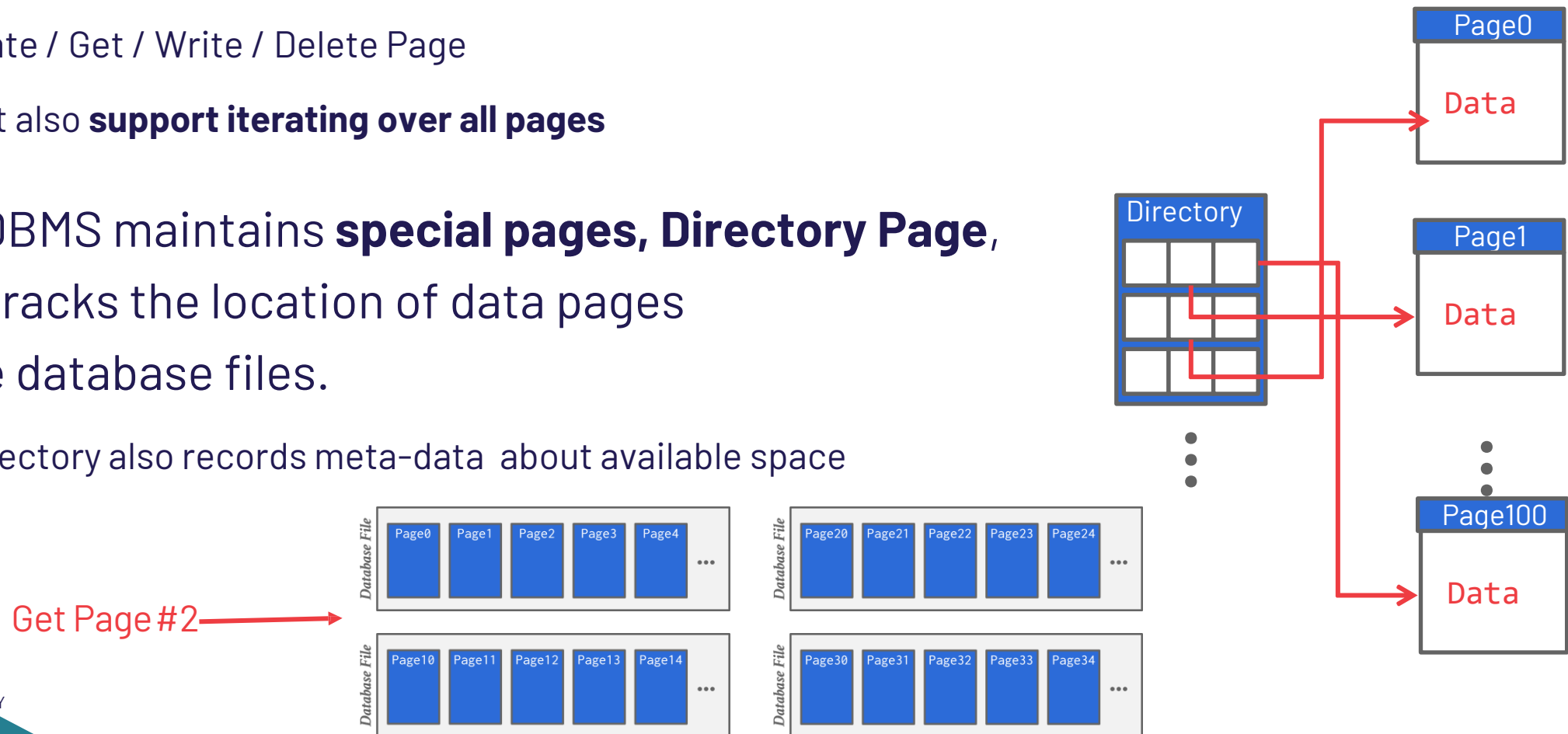
A heap file is an **unordered collection of pages** (with tuples inside them) that are stored in random order.

→ Create / Get / Write / Delete Page

→ Must also **support iterating over all pages**

The DBMS maintains **special pages, Directory Page**, that tracks the location of data pages in the database files.

The directory also records meta-data about available space



Outline

1. Storage Architecture

- DBMS Stack ★
- Memory hierarchy ★
- Disk Oriented DBMS ★
- Sequential/Random access ★
- Page storage Architecture ★

2. Representation on Disk

- Page layout ★
- Tuple storage + tuple layout ★
- Row storage vs column storage ★
- Denormalized storage

3. Indexing

- Data Structures ★
- Assumption / requirements ★
- Types of indexes: Hash / Tree + Static / Dynamic ★
- Hash Table / Hash Index Variants ★
- Tree index ★
- Primary key and clustering ★
- Sparse / dense ★

4. When to use indexes & effect on queries

- selecting which index to build ★
- partial keys ★
- selecting which index is compatible with your query ★

★ **Core Concepts: you need to understand these**

Based on chapter & online slides:

12.1, 12.3, 12.6, 13.1-13.3, 14.1-14.3, 14.5

- from Database System Concepts 7th edition ; Silberschatz et al.

Intro to Database Systems (15-445/645), Andy Pavlo CMU

- <https://15445.courses.cs.cmu.edu/fall2022/>



Outline

1. Storage Architecture

- DBMS Stack ★
- Memory hierarchy ★
- Disk Oriented DBMS ★
- Sequential/Random access ★
- Page storage Architecture ★

2. Representation on Disk

- Page layout ★
- Tuple storage + tuple layout ★
- Row storage vs column storage ★
- Denormalized storage

3. Indexing

- Data Structures ★
- Assumption / requirements ★
- Types of indexes: Hash / Tree + Static / Dynamic ★
- Hash Table / Hash Index Variants ★
- Tree index ★
- Primary key and clustering ★
- Sparse / dense ★

4. When to use indexes & effect on queries

- selecting which index to build ★
- partial keys ★
- selecting which index is compatible with your query ★

★ **Core Concepts: you need to understand these**

Based on chapter & online slides:

12.1, 12.3, 12.6, 13.1-13.3, 14.1-14.3, 14.5

- from Database System Concepts 7th edition ; Silberschatz et al.

Intro to Database Systems (15-445/645), Andy Pavlo CMU

- <https://15445.courses.cs.cmu.edu/fall2022/>





Tuple Storage (I)

How can we store our data (tuples!) in a page?

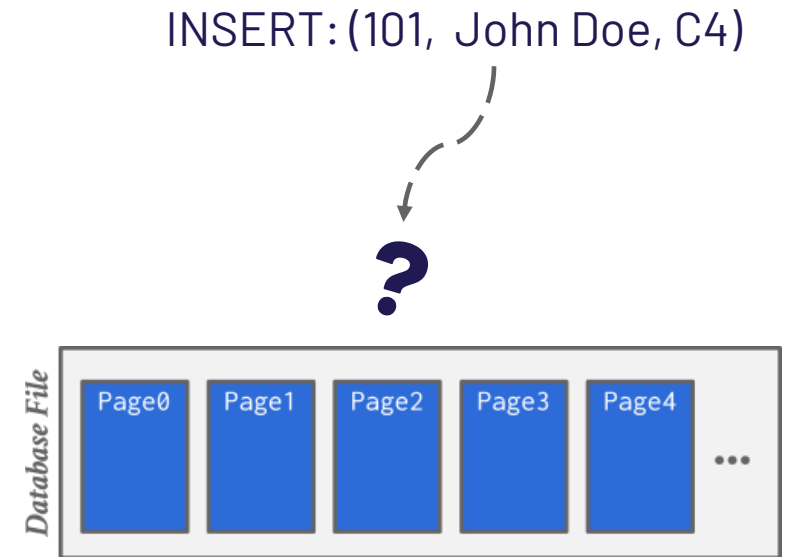
A tuple is a sequence of bytes

First find a page with enough space, then...?

Wrong idea: Just append tuples to a file!

Will cause problems when deleting tuples!

Also, what happens if we have some attribute with "varchar(1000)" ?



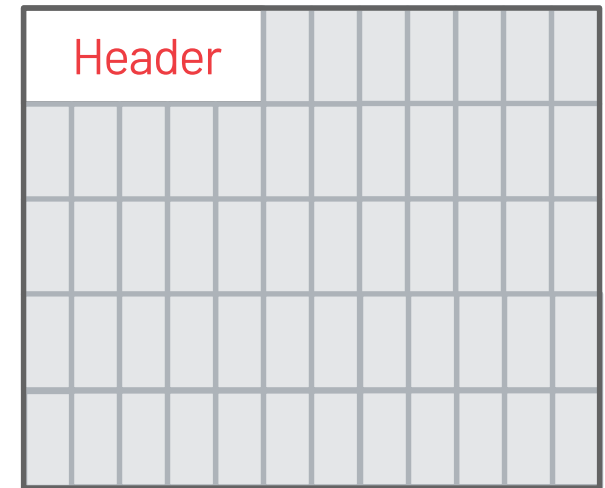
Remember: Special Pages can contain other types of auxiliary data, but for now we focus on tuples unless specified



Tuple Storage (II) - Slotted Pages

How can we store our data (tuples!) in a page?

1. **Decompose** a page in slots

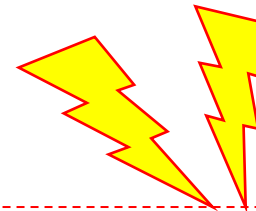
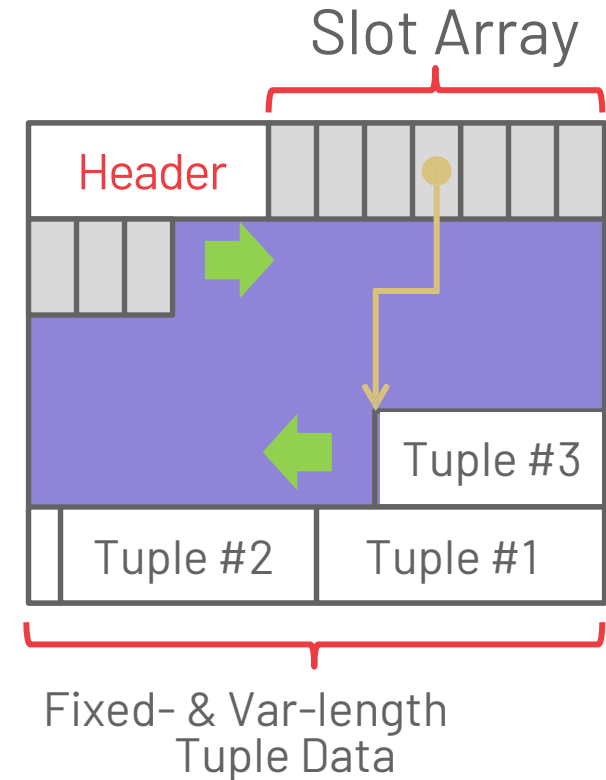




Tuple Storage (II) - Slotted Pages

How can we store our data (tuples!) in a page?

1. **Decompose** a page in slots
2. A **"slot array"** keeps indexes of the starting slot index and number of slots size of a given tuple
3. A **header** keeps track of
 - a. Number of slots used
 - b. Offset of last slot used



Tuples are assigned internal IDs (e.g., $\text{page_id} + \text{slot offset}$), in some systems you can access those, but should not rely on them



Tuple Layout

Fixed size records: all records use the same length, no matter if they use the space or not

101	J	o	h	n	_	D	o	e			C	4
102	J	a	n	e	_	D	o	l	l	y		

With fixed size records it is easier to know where to jump for record i , is just a multiplication $i * \text{size}$

Variable size records: records can have different sizes, need to store the “length” of each attribute, mapping also NULLs, uses a **tuple header**

0	1	9	000	101	J	o	h	n	_	D	o	e	C	4
0	1	0	001	102	J	a	n	e	_	D	o	l	l	y

For very large attributes one uses “overflow” pages

Tuple Header

Null Bitmap

Tuple Header is usually present also in Fixed Size records

```
CREATE TABLE member(  
    pid integer PRIMARY KEY,  
    name varchar(10) NOT NULL,  
    rank char(2)  
);  
Insert:  
(101, John Doe, C4)  
(102, Jane Dolly)
```



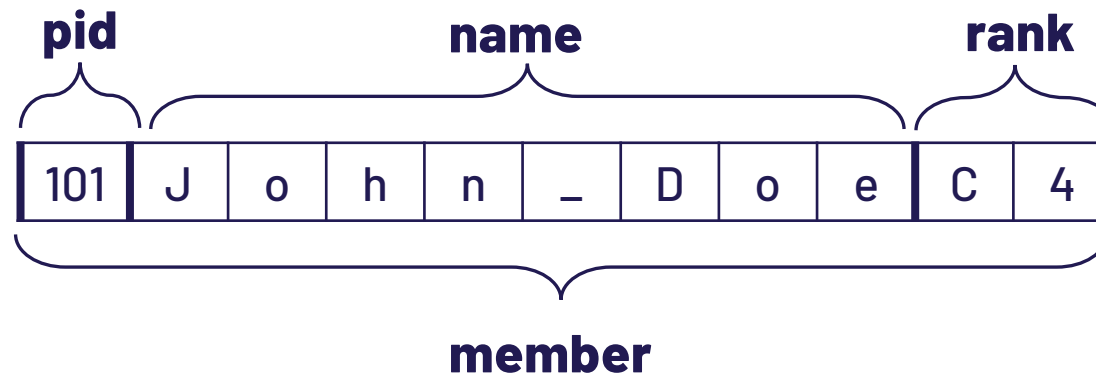
Alternative Storage Layouts

Observation: the relational model does not specify that the DBMS must store all of a tuple attributes together!!!

Up to here we assumed the **n-ary storage model: row storage**

```
CREATE TABLE member(  
  pid integer PRIMARY KEY,  
  name varchar(10) NOT NULL,  
  rank char(2)  
);
```

1 record = 1 tuple



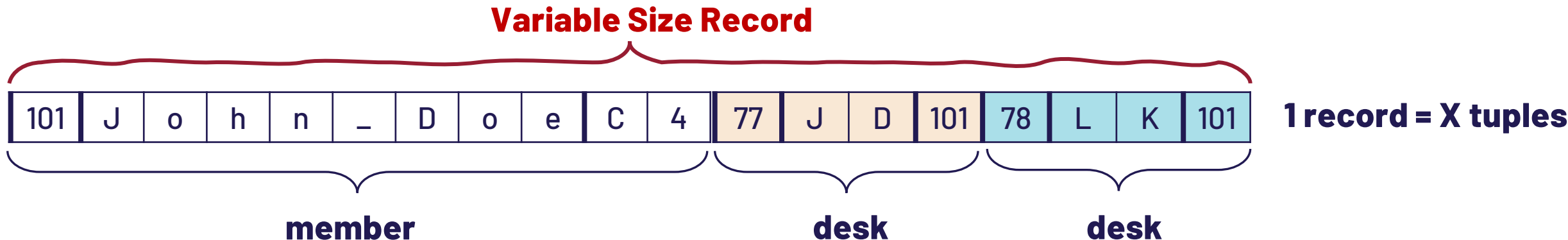
Denormalized Tuples

Use **variable size records** to store related tuples

Goal: Speed up joins by storing together tuples that are accessed together.

This is called **physical denormalization** (pre join)

```
CREATE TABLE member(  
    pid integer PRIMARY KEY,  
    name varchar(10) NOT NULL,  
    rank char(2)  
);  
CREATE TABLE desk(  
    did integer PRIMARY KEY,  
    label char(2),  
    pid integer,  
    REFERENCES member(pid)  
);
```

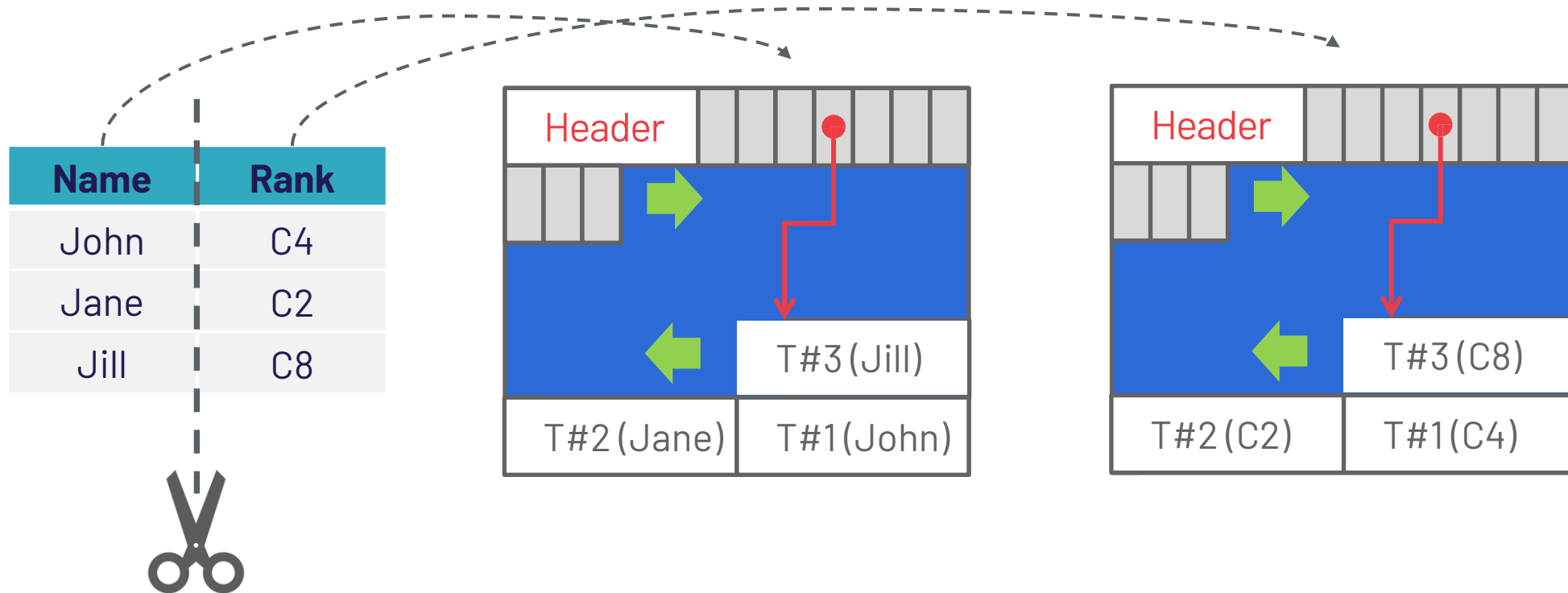


Page Header & Tuple Header need to keep track of this information!



Decomposition Storage Model

The DBMS stores the values of a **single attribute for "all" tuples** contiguously in a single page: **column store**



For "ANALYTICAL" workloads, this is what you want to use

OLTP and **OLAP** refer to different kinds of use cases & systems.

Online transaction processing (OLTP) captures, stores, and processes data from transactions in real time. **Online analytical processing (OLAP)** uses complex queries to analyze aggregated historical data from OLTP systems.



Page-Oriented Architecture (Disk Based)

Insert a new tuple:

1. Check page directory to find a page with a free slot
2. Retrieve the page from disk (if not in memory – buffer pool – already)
3. Check slot array to find empty space in the page

Delete / Update an existing Tuple

1. Check page directory to find location of page
2. Retrieve the page from disk (if not in memory – buffer pool – already)
3. Find offset in page using slot array
4. Overwrite existing data (if new data fits...)

For Insert/Delete/Update-Intensive workloads
this architecture can be quite slow

Log-Structured Storage overcomes some of these problems:
we only store an incremental record of PUT/DELETE
operations (not covered here – used mostly for different
systems: key-value stores)

Outline

1. Storage Architecture

- DBMS Stack ★
- Memory hierarchy ★
- Disk Oriented DBMS ★
- Sequential/Random access ★
- Page storage Architecture ★

2. Representation on Disk

- Page layout ★
- Tuple storage + tuple layout ★
- Row storage vs column storage ★
- Denormalized storage

3. Indexing

- Data Structures ★
- Assumption / requirements ★
- Types of indexes: Hash / Tree + Static / Dynamic ★
- Hash Table / Hash Index Variants ★
- Tree index ★
- Primary key and clustering ★
- Sparse / dense ★

4. When to use indexes & effect on queries

- selecting which index to build ★
- partial keys ★
- selecting which index is compatible with your query ★

★ **Core Concepts: you need to understand these**

Based on chapter & online slides:

12.1, 12.3, 12.6, 13.1-13.3, 14.1-14.3, 14.5

- from Database System Concepts 7th edition ; Silberschatz et al.

Intro to Database Systems (15-445/645), Andy Pavlo CMU

- <https://15445.courses.cs.cmu.edu/fall2022/>



Outline

1. Storage Architecture

- DBMS Stack ★
- Memory hierarchy ★
- Disk Oriented DBMS ★
- Sequential/Random access ★
- Page storage Architecture ★

2. Representation on Disk

- Page layout ★
- Tuple storage + tuple layout ★
- Row storage vs column storage ★
- Denormalized storage

3. Indexing

- Data Structures ★
- Assumption / requirements ★
- Types of indexes: Hash / Tree + Static / Dynamic ★
- Hash Table / Hash Index Variants ★
- Tree index ★
- Primary key and clustering ★
- Sparse / dense ★

4. When to use indexes & effect on queries

- selecting which index to build ★
- partial keys ★
- selecting which index is compatible with your query ★

★ **Core Concepts: you need to understand these**

Based on chapter & online slides:

12.1, 12.3, 12.6, 13.1-13.3, 14.1-14.3, 14.5

- from Database System Concepts 7th edition ; Silberschatz et al.

Intro to Database Systems (15-445/645), Andy Pavlo CMU

- <https://15445.courses.cs.cmu.edu/fall2022/>



DBMS Internals (II)

- We have seen the basic storage mechanism of tuples on disk → **how we persist data!**
- Next, we see **how the DBMS supports queries!**
this includes both read and write from/to pages
- Tuple Data is access via **specialized Data Structures:**
both temporary and permanent data structures that keep data organized

Query Planning

Operator Execution

Access Methods

Buffer Pool Manager

Disk Manager

Logical layer

CREATE TABLE ...
SELECT ...

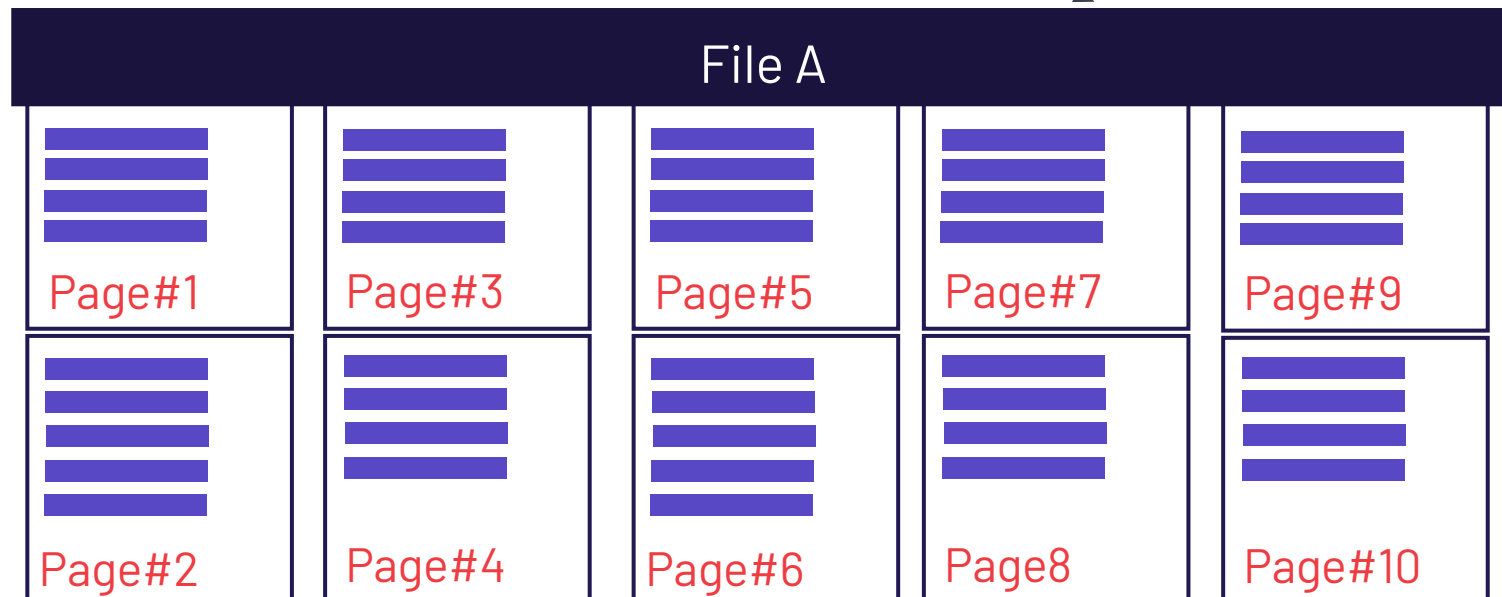
Name	Rank	Grade
Jens	C3	12
Zack	C4	7
Jane	C2	10
July	C6	6
Jill	C8	4
...	...	...

How do I find all the records
with Rank='C3' ??



Physical Layer

Files,
Pages,
Records,
Fields



Logical layer

CREATE TABLE ...
SELECT ...

Name	Rank	Grade
Jens	C3	12
Zack	C4	7
Jane	C2	10
July	C6	6
Jill	C8	4
...	...	...

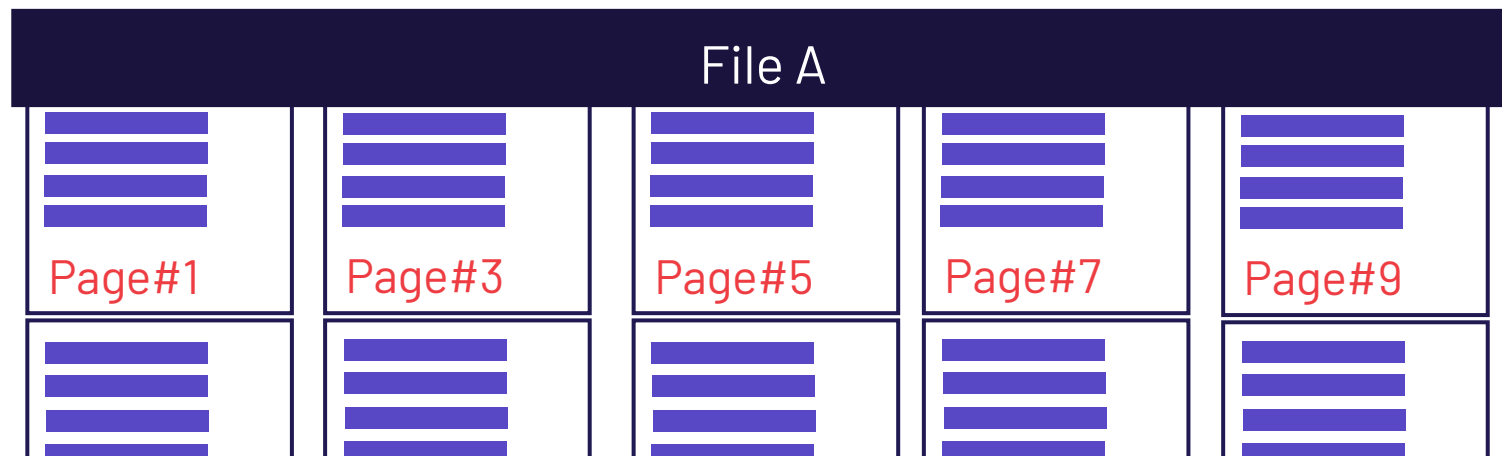
How do I find all the records
with Rank='C3' ??

Data
Structure



Physical Layer

Files,
Pages,
Records,
Fields



Where do I put it?

SQL Query

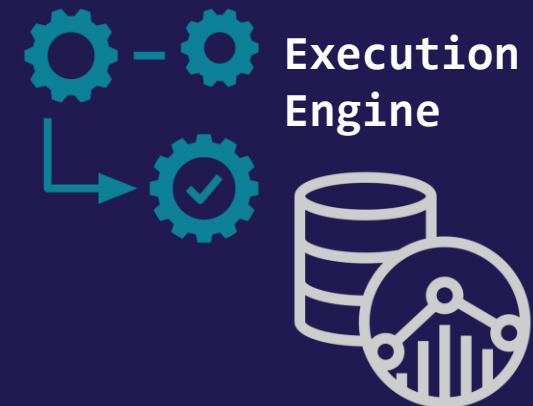


```
SELECT p.id, COUNT(*)  
FROM professor p JOIN course c  
      ON p.id = c.taughtby  
WHERE p.rank > ALL (SELECT rank  
                    FROM professor)  
GROUP BY p.id ;
```

Intermediate
Result

Intermediate
Result

Intermediate
Result





Data Structures

Data Structures are **used for many purposes** in different components of the system.

- Internal Meta-data
- Core Data Storage
- Temporary Data Structures
- Table Indexes

For example:

To compute faster JOINS & GROUP BY of intermediate results!

Quicker access to tuples on disk!
...one must keep track of multi-threaded access ...



Requirements

GOAL: Access as little data as possible

- There are many rows (records, tuples, ...)
- We usually require to access only few of them (e.g., WHERE id = 42)
- We need to support: PUT / GET / DELETE

Dynamic vs. Static Data Structures (DS)



“Do we know how much data we expect to handle?”

If we expect it to grow and shrink a lot, we use a dynamic DS, otherwise we use **a static data structure**.

Static implies we allocate memory, if we fill it, we have to start from scratch!

With a static DS, if you guess right, you will be faster than if you use a dynamic DS, but if you guess wrong, you pay the price! (Can we really guess...? See next lectures!)



Hash Tables & Trees

We are going to see **2 types of data structures**:

- **Hash Tables**

- **Unsorted Access**, fast for get/put single <key:value> pairs: e.g., what is the tuple with primary key =42

- **Trees (B+Trees)**

- **Sorted Access**, can support also range queries, e.g., the tuples with attribute ≥ 42



Hash Tables

Hash tables are **unordered associative arrays that map keys to values**

Given a Hash Function with **input:** values as bytes → **output:** integer

→ **the output integer is used as “index”** in an array where to find the value

Space Complexity: **$O(n)$**

Time Complexity:

- Average **$O(1)$**
- Worst **$O(n)$**

Also constants matters!
Sometimes we say the
access time is ~ 1.2

PUT <key, value>

PUT <Jane, C6>

$$\text{hash}(\text{Jane}) \% 6 = 116 \% 6 = 2$$

fixed size record

	Name	Rank
0		
1	Zack	C4
2	<u>Jane</u>	<u>C6</u>
3		
4	Jill	C8
5	Alex	C3



Hash Table Decisions

1. Hash Function: Facebook XXHash (2012)

1. We do not assume a perfect hash function
2. We do not have infinite space
3. It should be fast and still incur in few collisions

2. Hashing Scheme

1. What happens when we have collisions
2. What happens when the table is full

Static: Linear Probe / Robin Hood / Cuckoo

Dynamic: Chained Hashing / Extendible / Linear ($\neq$ Probe)

Collision!

hash(Max) = 2

PUT <Max, C2>

	Name	Rank
0		
1	Zack	C4
2	Jane	C2
3		
4	Jill	C8
5	Alex	C3



Linear Probe Hashing -PUT

PUT <key,value>:

1. Hash <key>
2. if empty, insert entire <key,value> pair
or if not-empty (collision!) insert in next available slot!

Time Complexity:

- Average $O(1) \sim 1.2$
- Worst $O(n)$

*sometimes we will do
more work than $O(1)$*

Collision!

hash(Max) = 2

PUT <Max, C2>

	Name	Rank
0		
1	Zack	C4
2	Jane	C2
3	<u>Max</u>	<u>C2</u>
4	Jill	C8
5	Alex	C3



Linear Probe Hashing -PUT (II)

PUT <key,value>:

1. Hash <key>
2. if empty, insert entire <key,value> pair
or if not-empty (collision!) insert in next available slot!

It's called also closed hashing

Time Complexity:

- Average $O(1)$
- Worst $O(n)$

Worst case linear scan
also in insert

Collision!

?

hash(Dave) = 2

PUT <Dave, C6>

	Name	Rank
0	<u>Dave</u>	<u>C6</u>
1	Zack	C4
2	Jane	C2
3	Max	C2
4	Jill	C8
5	Alex	C3

circular buffer



Linear Probe Hashing -GET

PUT <key,value>:

1. Hash <key>
2. if empty, insert entire <key,value> pair
or if not-empty (collision!) insert in next available slot!

GET <key>

1. Hash <key>
2. if empty, return "not found"
if not empty, compare <key> with slot
if same, return <value>
if not same, search until find an empty slot
if empty slot: return "not found"

hash(Jeff) = 1

GET <Jeff> → ∅

We return "not found" only if we reach an empty bucket

	Name	Rank
0		
1	Zack	C4
2	Jane	C2
3		
4	Jill	C8
5	Alex	C3



Linear Probe Hashing -DELETE

PUT <key,value>:

1. Hash <key>
2. **if empty, insert** entire <key,value> pair
or **if not-empty** (collision!) insert in **next available slot!**

DELETE <key>:

1. **GET** <key>
2. **if key is present** PUT <key, tombstone>
DELETE <Max>

GET <key>

1. Hash <key>
2. **if empty**, return "not found"
if not empty, compare <key> with slot
if same, return <value>
if not same, search until find an empty slot
if empty slot: return "not found"

hash(Jeff) = 1

GET <Jeff> → C7

	Name	Rank
0		
1	Zack	C4
2	Jane	C2
3	Max	C2
4	Jeff	C7
5	Alex	C3

Actually: We return "not found" only if we reach an empty bucket or if we find the key with a tombstone as value

Records with tombstones are considered full during search but can be reused and overwritten!



Linear Probe Hashing – Non-unique Keys

The approach we just saw here assumed only “unique keys”

If two records with the same key are inserted, the last one will overwrite the previous!

PUT <Zack,C4> and then PUT <Zack,C8> will result in just <Zack,C8>



*This happens every time we use a **hash table** in memory to **execute a join faster** (hash join, more in the next lectures)*

Solutions:

1. Separate Linked List: a separate storage area for “**overflow**” records

Also called Open Hashing

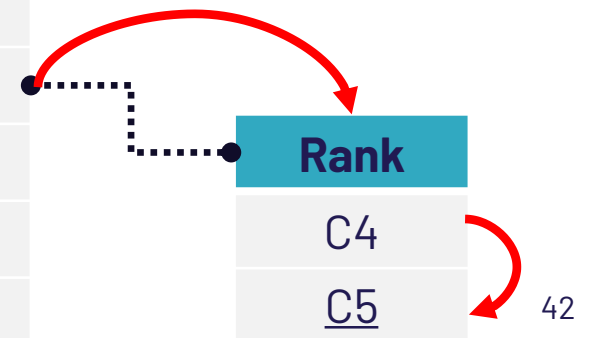
PUT <Jane, C5>

hash(Jane) = 2

Open Hashing

	Name	Rank
0		
1	Zack	C4
2	Jane	C2
3		
4	Jill	C8
5	Alex	C3

Often you just store the keys, and all values go in the external linked list





Linear Probe Hashing – Non-unique Keys

The approach we just saw here assumed only “unique keys”

If two records with the same key are inserted, the last one will overwrite the previous!



PUT <Zack,C4> and then PUT <Zack,C8> will result in just <Zack,C8>

Solutions:

1. **Separate Linked List:** a separate storage area for “**overflow**” records

Also called Open Hashing

2. **Redundant Keys:** Assume you never overwrite & store duplicate keys with their entries

Simpler to implement, most used

PUT <Jane, C5>

hash(Jane) = 2

Redundant keys		
	Name	Rank
0		
1	Zack	C4
2	Jane	C2
3	Jill	C8
4	<u>Jane</u>	<u>C5</u>
5	Alex	C3



Hash Table Decisions

1. Hash Function: Facebook XXHash (2012)

1. We do not assume a perfect hash functions
2. We do not have infinite space
3. It should be fast and still incur in few collisions

2. Hashing Scheme

*Now we see how to handle
a dynamically growing set
of records*

1. What happens when we have collisions
2. What happens when the table is full

Static: Linear Probe / Robin Hood / Cuckoo

Dynamic: Chained Hashing / Extendible / Linear ($\neq$ Probe)

Static hashing schemes **require** the system to **rebuild** the table **if no space is left**.
Dynamic Hash Tables solved this!
(but there is a price to pay)



Collision!

?

hash(Max) = 2

PUT <Max, C2>

	Name	Rank
0		
1	Zack	C4
2	Jane	C2
3		
4	Jill	C8
5	Alex	C3

Chained Hashing -PUT

For Dynamic Input data

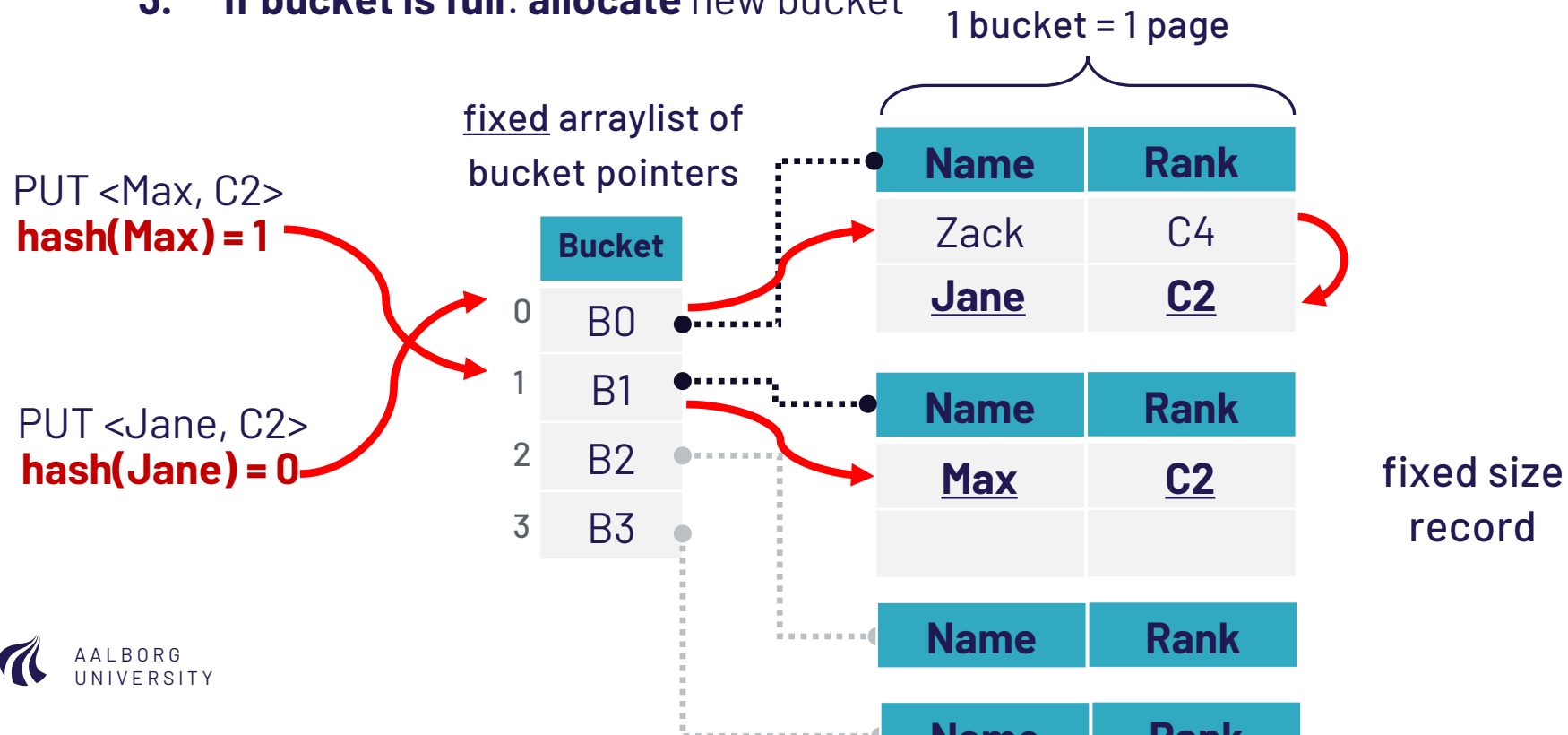
PUT <key,value>:

1. **Hash** <key> → **find bucket**
2. **if bucket has free space: append** record
3. **if bucket is full: allocate** new bucket

Time Complexity:

- Average $O(1) \sim 1.2$
- Worst $O(n)$

Often, we will do more work than $O(1)$



Chained Hashing -PUT

PUT <key,value> :

1. Hash <key> → find bucket
2. if bucket has free space: **append** record
3. if bucket is full: **allocate** new bucket

Time Complexity:

- Average $O(1) \sim 1.2$
- Worst $O(n)$

*if we are very unlucky,
all records are in only
one bucket*

PUT <Alex, C4>
hash(Alex) = 0

**Search follows the
same steps of PUT**

*We can keep meta-data in
the bucket list to avoid some search*

fixed arraylist of
bucket pointers

	Bucket
0	B0
1	B1
2	B2
3	B3

1 bucket = 1 page

Name	Rank
Zack	C4
Jane	C2

Name	Rank
Max	C2

Name	Rank
------	------

Name	Rank
------	------

*When a bucket is full,
we allocate an overflow bucket*

Name	Rank
<u>Alex</u>	<u>C4</u>

Bucket chain can grow with new buckets:
all buckets are of the same fixed size

Because the arraylist of **bucket pointers never changes in size**... some **bucket chain can grow** very long! Search performances will suffer



Extendible Hashing

Chained-hashing approach where we **split buckets** instead of letting the linked list grow forever.

Multiple **slot locations can point to the same bucket** chain.

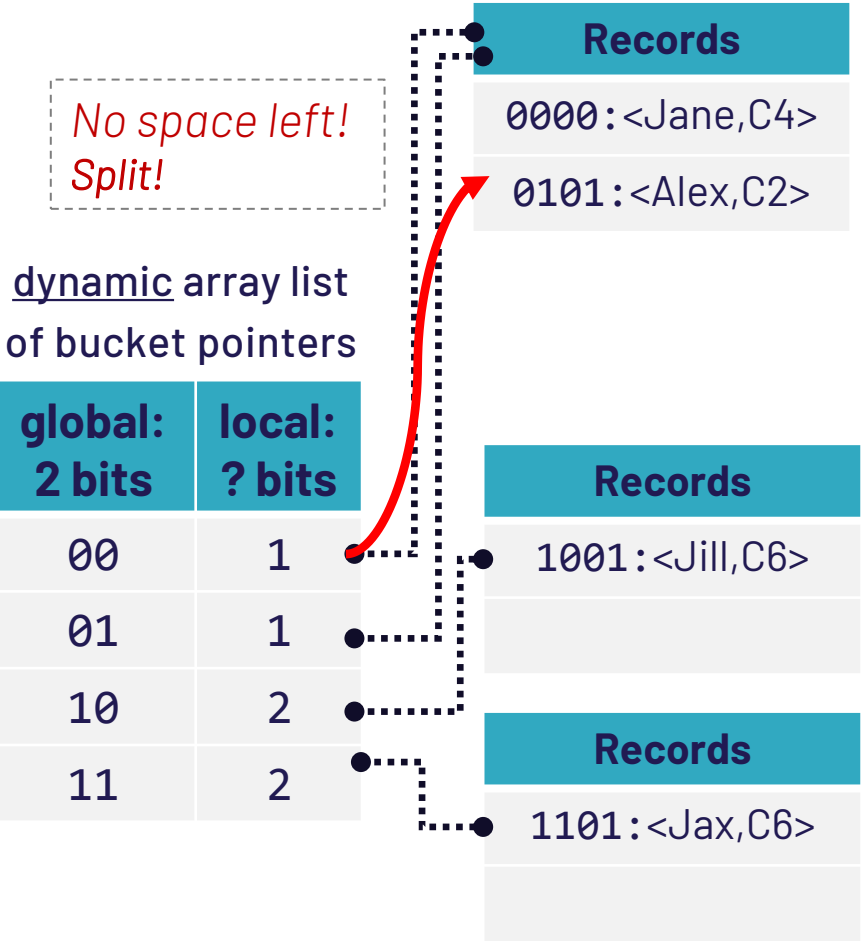
→ Given the same hash value, we consider different “portions” of it to decide which bucket to use

Reshuffle bucket entries on split and increase the number of bits to examine.

→ Data movement is localized to just the split chain.

hash(Max) = 3 = 0011
PUT <Max, C2>

In chained hashing, because **the arraylist of bucket pointers never changes** in size... some bucket **chain can grow very long!** Search performances will suffer



Extendible Hashing

Chained-hashing approach where we **split buckets** instead of letting the linked list grow forever.

Multiple **slot locations can point to the same bucket** chain.

→ Given the same hash value, we consider different “portions” of it to decide which bucket to use

Reshuffle bucket entries on split and increase the number of bits to examine.

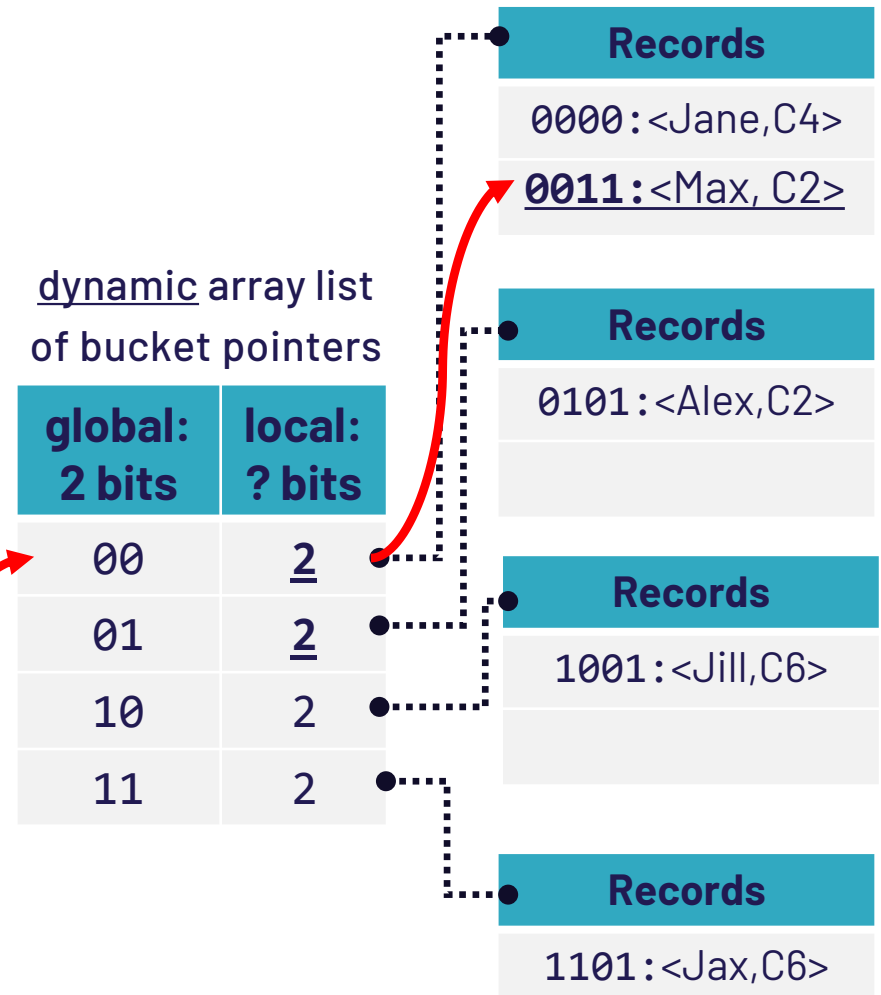
→ Data movement is localized to just the split chain.

The arraylist of bucket pointers can also expand when we consider more bits

hash(Max) = 3 = **00**11
PUT <Max, C2>

In chained hashing, because **the arraylist of bucket pointers never changes** in size... some bucket **chain can grow very long!** Search performances will suffer

!





Hash Tables

Hash tables are unordered associative arrays that map keys to values

Ideal Space Complexity: $O(n)$

→ When we allocate space, we need to consider Static vs. Dynamic schemes

Average runtime for GET/PUT/DELETE $O(1)$

→ Yet, there will be extra operations

Worst runtime $O(n)$

→ See Static vs. Dynamic consideration to avoid this

Frequently used for one-shot internal indexing (built on the spot) during query execution. Very rarely used for “on disk” persistent indexing, but still applicable.

Logical layer

CREATE TABLE ...
SELECT ...

Name	Rank	Grade
Jens	C3	12
Zack	C4	7
Jane	C2	10
July	C6	6
Jill	C8	4
...	...	...

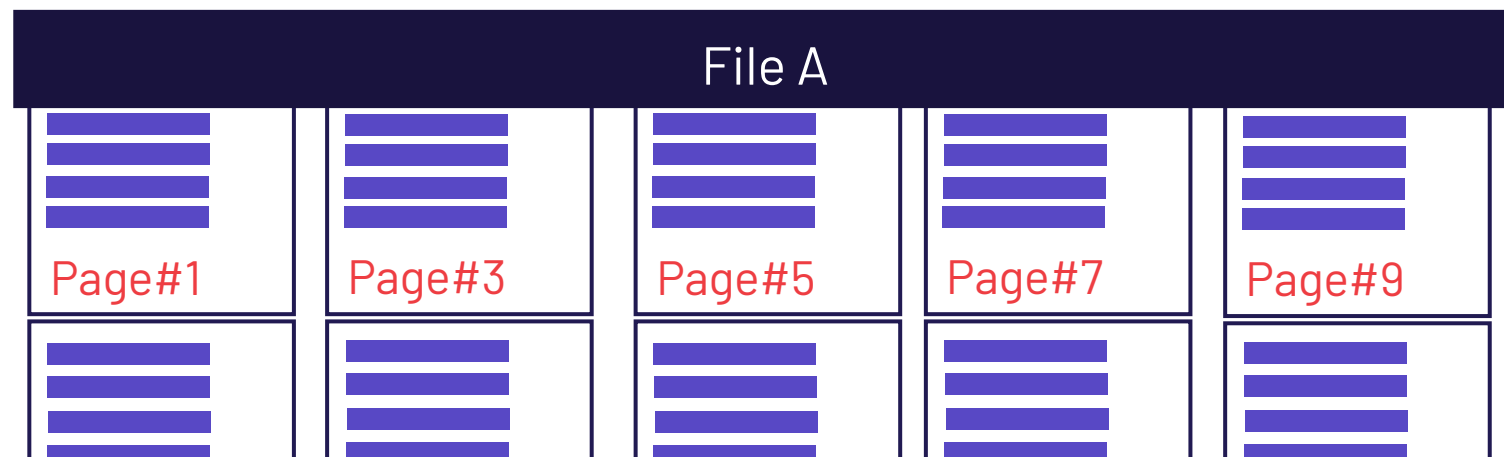
How do I find all the records
with Rank='C3' or Grade > 6 ??

*The DBMS needs to have an index to satisfy the workload
and avoid when possible, a sequential scan of all pages*



Physical Layer

Files,
Pages,
Records,
Fields





Data Structures

Data Structures are **used for many purposes** in different components of the system.

- Internal Meta-data
- Core Data Storage
- Temporary Data Structures
- Table Indexes



Quicker access to tuples on disk!
...one must keep track of multi-threaded access ...



Table Indexes

Auxiliary Data Structures, contains **a replica** of all or a subset of a table's attributes

They are stored in a specialized data structures
(sorted/unsorted)

Allow **efficient access to the tuples** based on the value of the attributes for which it was built

It is the job of the DBMS to ALWAYS ensure that accessing the index allows to access the same information contained in the DB.



Table Indexes (II)

The **DBMS is demanded to “pick”** the best index **available** to execute a query efficiently
(more on this in future lecture)

There is always a tradeoff:

- more indexes → easier to run **efficient queries**
- more indexes → **more space** required on **disk**
- more indexes → **more space** occupied in **main memory**
- more indexes → **more work** to do when insert/delete/update



Table Indexes (II)

The **DBMS** is demanded to “**pick**” the best index **available** to execute a query efficiently
(more on this later)

There is always a tradeoff:

- more indexes → easier to run **efficient queries**
- more indexes → **more space** required on **disk**
- more indexes → **more space** occupied in **main memory**
- more indexes → **more work** to do when insert/delete/update

Deciding which index to build is not always an obvious choice...

Do we want to speed up average queries or slowest queries?

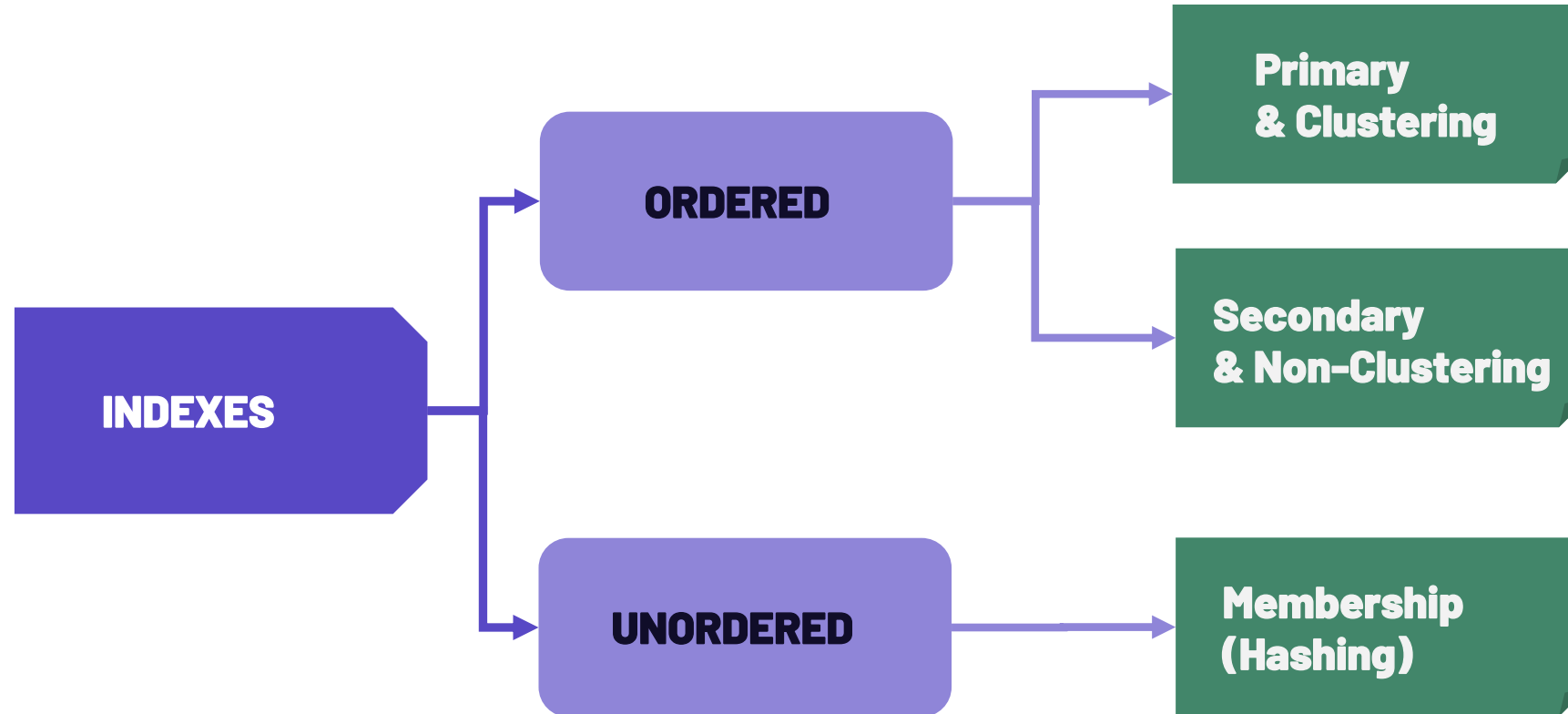
(e.g., 90th percentile)





INDEXES:

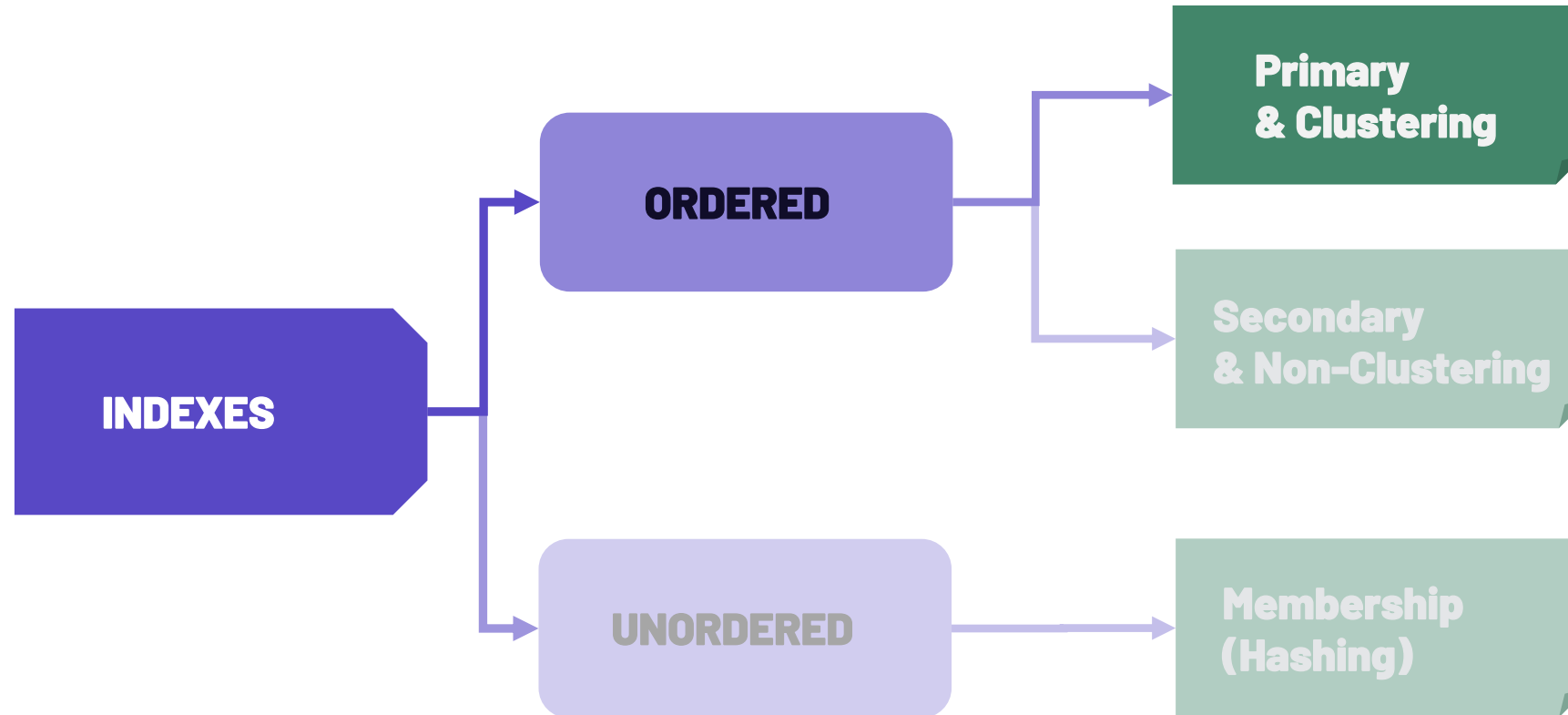
Find Locations Of Records By Accessing As Little Data As Possible





INDEXES:

Find Locations Of Records By Accessing As Little Data As Possible





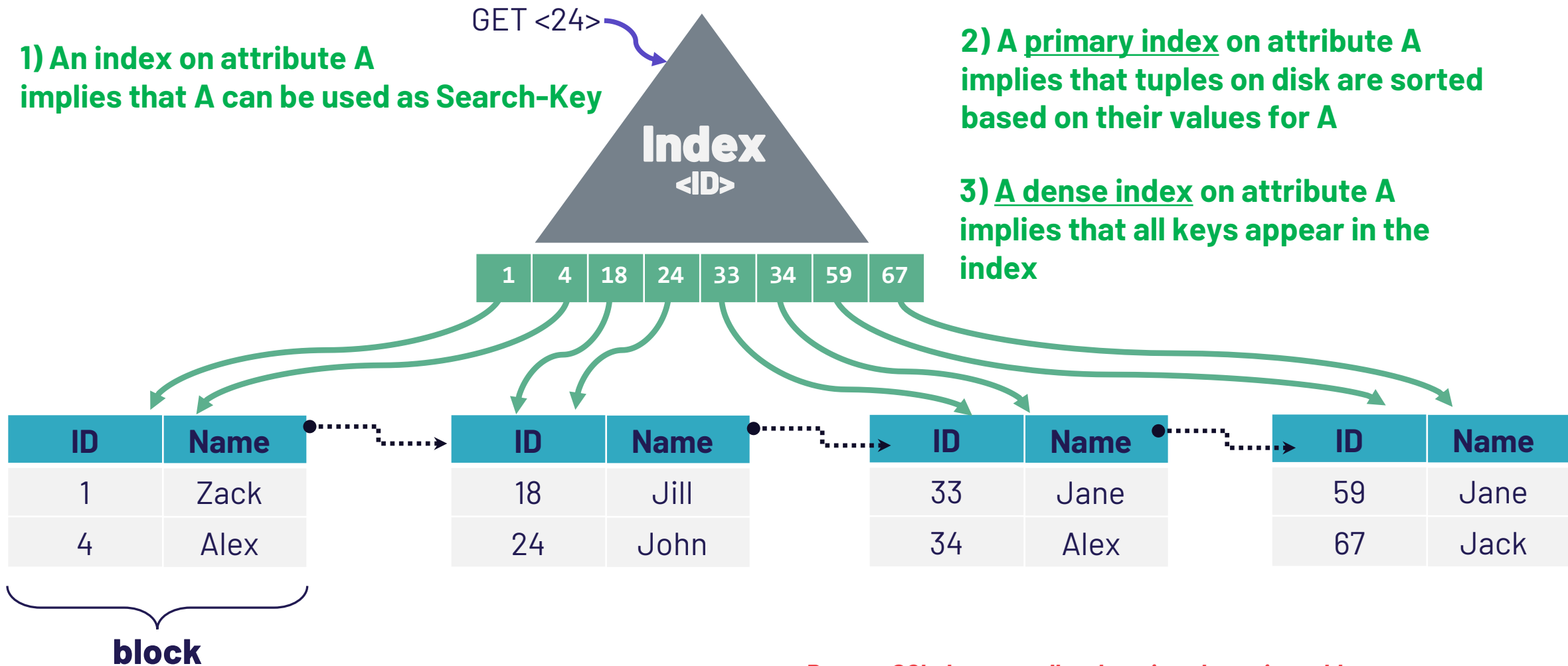
Primary Dense Index

Define the ordering & location of the tuples on disk

1) An index on attribute A implies that A can be used as Search-Key

2) A primary index on attribute A implies that tuples on disk are sorted based on their values for A

3) A dense index on attribute A implies that all keys appear in the index





Primary Sparse Index

Define the ordering & location of the tuples on disk

1) An index on attribute A implies that A can be used as Search-Key

A search key is not necessarily a primary key or a unique key.



GET <24>

Index
<ID>

1 18 33 59 78 95

2) A primary index on attribute A implies that tuples on disk are sorted based on their values for A

3) A sparse index on attribute A implies that it contains only one key per block

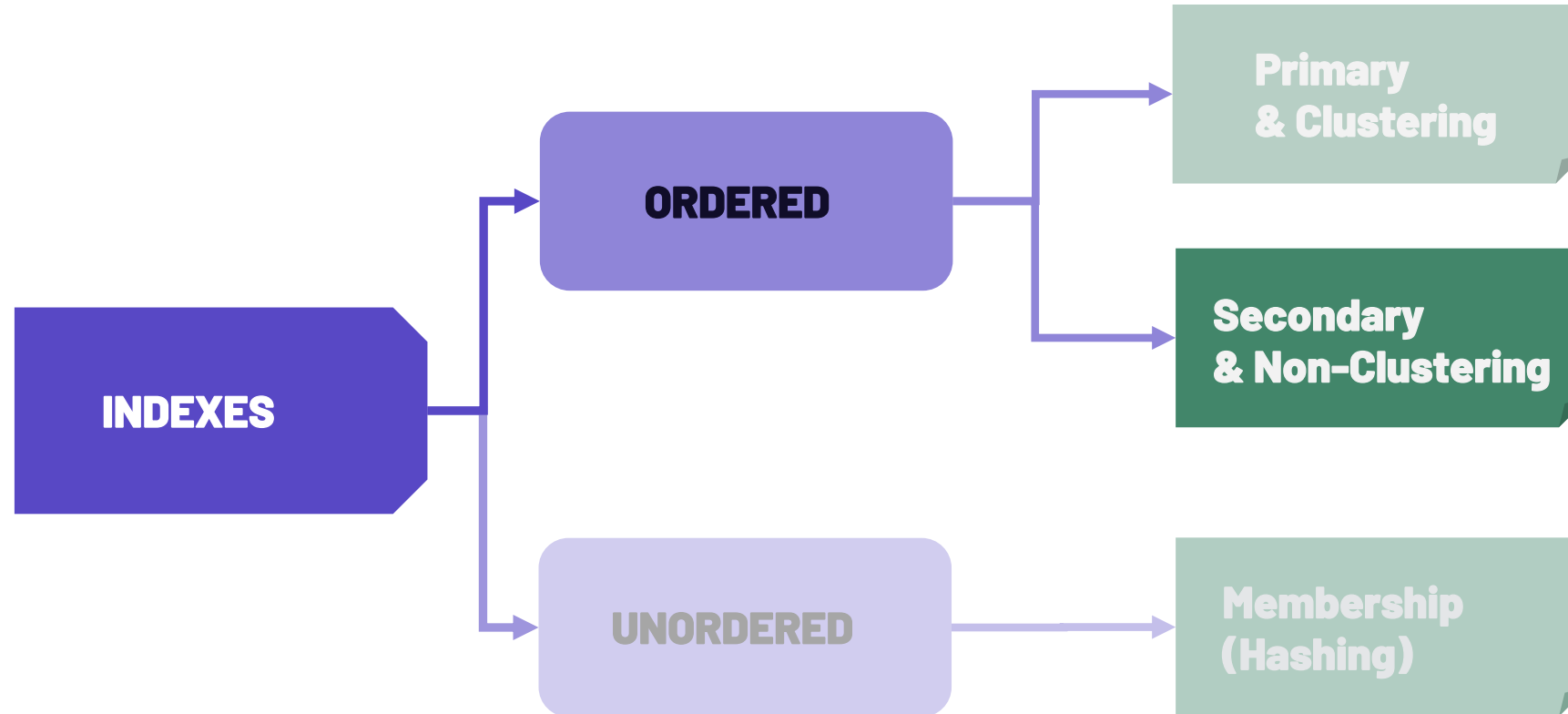
ID	Name	ID	Name	ID	Name	ID	Name
1	Zack	18	Jill	33	Jane	59	Jane
4	Alex	24	John	34	Alex	67	Jack

block



INDEXES:

Find Locations Of Records By Accessing As Little Data As Possible





Secondary Dense Index

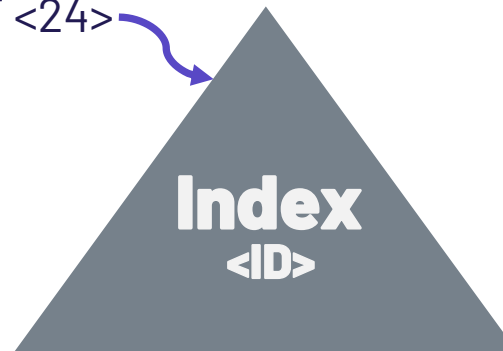
Define only the location of the tuples on disk

1) An index on attribute A implies that A can be used as Search-Key.

2) A secondary index on attribute A does not enforce any ordering on the tuples.

3) A secondary index on attribute A can only be dense, all values need to be indexed.

GET <24>



1	4	18	24	33	34	59	67
---	---	----	----	----	----	----	----

Here Clustering Attribute is "Name"

ID	Name
4	Alex
34	Alex

ID	Name
67	Jack
33	Jane

ID	Name
59	Jane
18	Jill

ID	Name
24	John
1	Zack

block



Indirect Multi-level Index

Combine a Sparse Index on top of a Dense Index

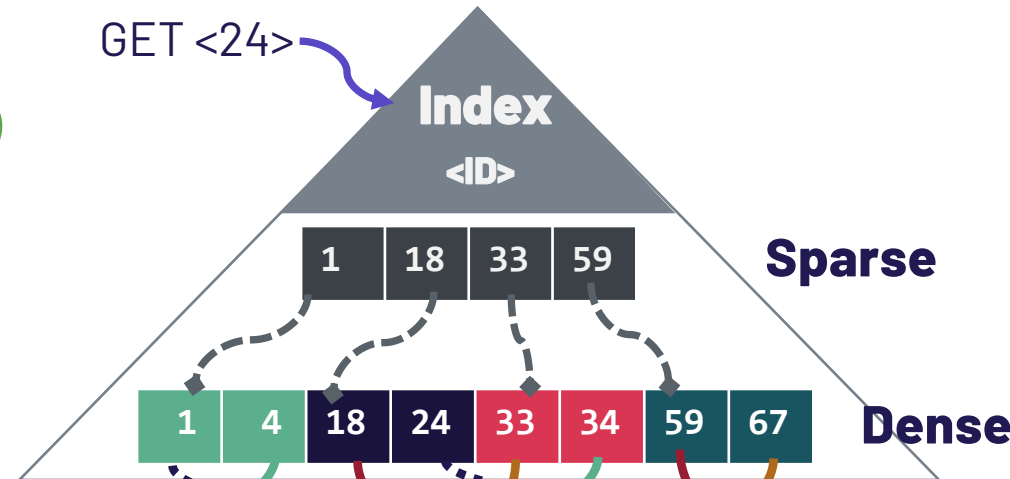
Goal: the outer index (sparse) fits into main memory

The dense index is divided in blocks

The sparse index points to blocks of the dense index

The dense index points to the actual blocks of records.

The number of levels can be greater than two



Here Clustering Attribute is "Name"

ID	Name	ID	Name	ID	Name	ID	Name
4	Alex	67	Jack	59	Jane	24	John
34	Alex	33	Jane	18	Jill	1	Zack

block



Primary / Secondary & Sparse / Dense Overview

Primary vs. Secondary:

Is Primary: If the tuples are ordered on the search key (also called clustering)

Dense vs. Sparse:

Is Dense: if all the keys appear in the index

Note that a sparse index should always allow to find all entries, but for this reason

only a primary index can be sparse!

Tradeoff

A dense index allows **faster location** of records since it contains direct pointers to all tuples, but requires more space

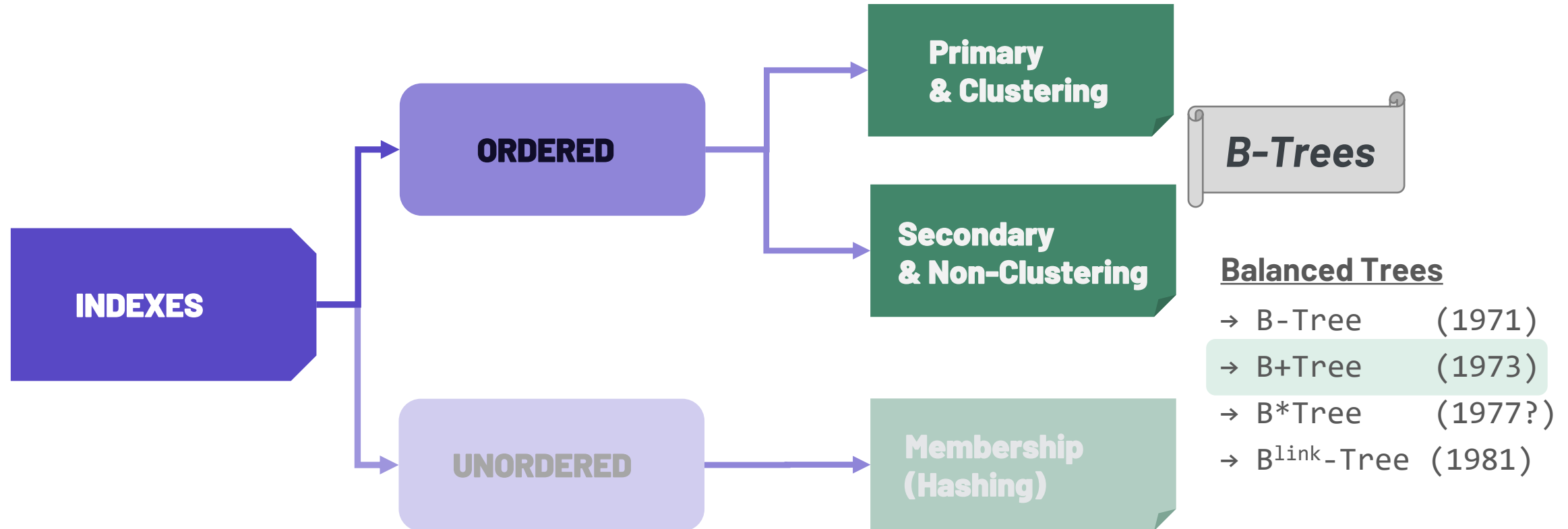
A sparse index has **smaller** memory requirement (more of it can fit in memory)





INDEXES:

Find Locations Of Records By Accessing As Little Data As Possible



B → as in Balanced, Boeing, Bayer ... ?



B+Tree: overview

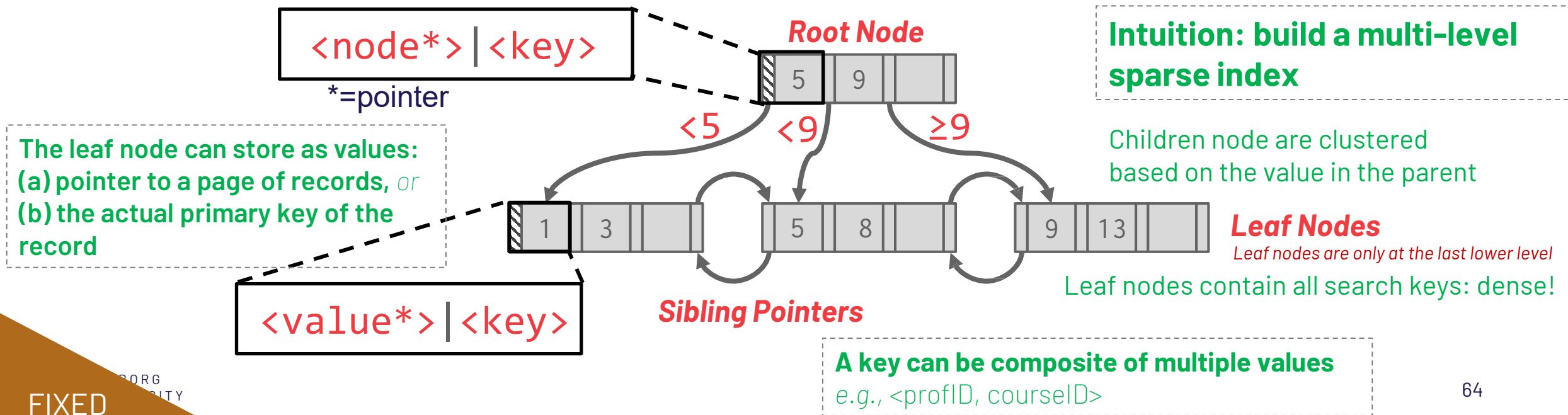
Self-balancing tree data structure: a multi-level search tree

A B+Tree Keeps data sorted on the search attribute

Allows search for values, sequential access, insertions, and deletions **always always in $O(\log N)$**

→ **Generalization of a binary search tree, since a node can have more than two children.**

→ **Optimized for systems that read and write large blocks of data.**





B+Tree: properties

Self-balancing tree data structure: a multi-level search tree

A B+Tree Keeps data sorted on the search attribute

A B+Tree is an M-way search tree with the following properties:

→ It is **perfectly balanced** (i.e., every leaf node is at the **same depth** in the tree)

→ **Every node** other than the root is **at least half-full**

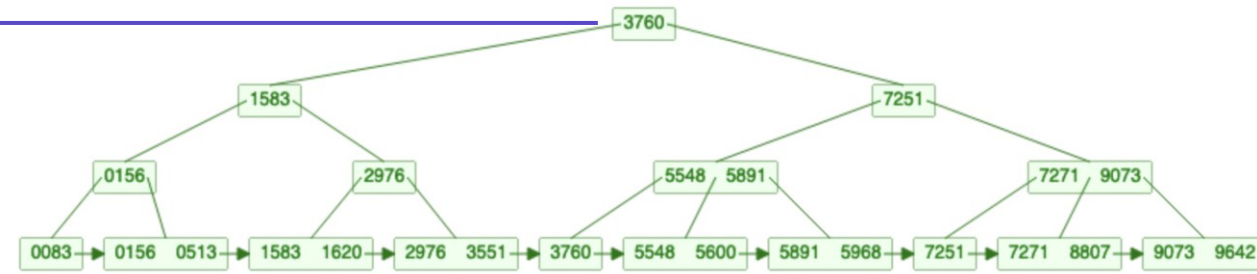
$$M/2 - 1 \leq (\text{num keys}) \leq M - 1$$

M is also called the fanout or degree of a node

→ Every **inner node with k(<M) keys** has **k+1** non-null **children**

The root node is the only one that can have as little as 2 keys only!

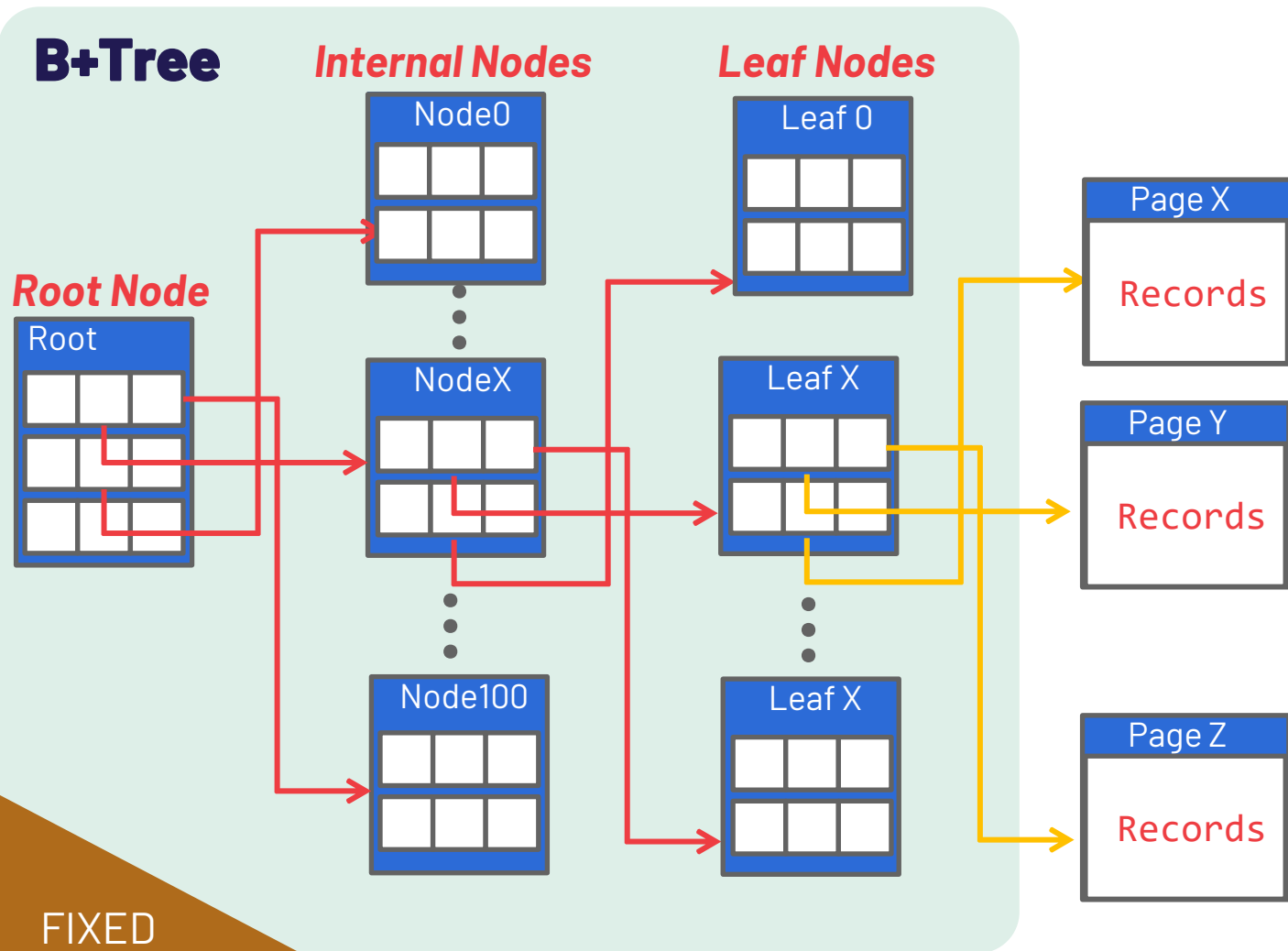
*From root to leaf, always 3 steps
depth=3*





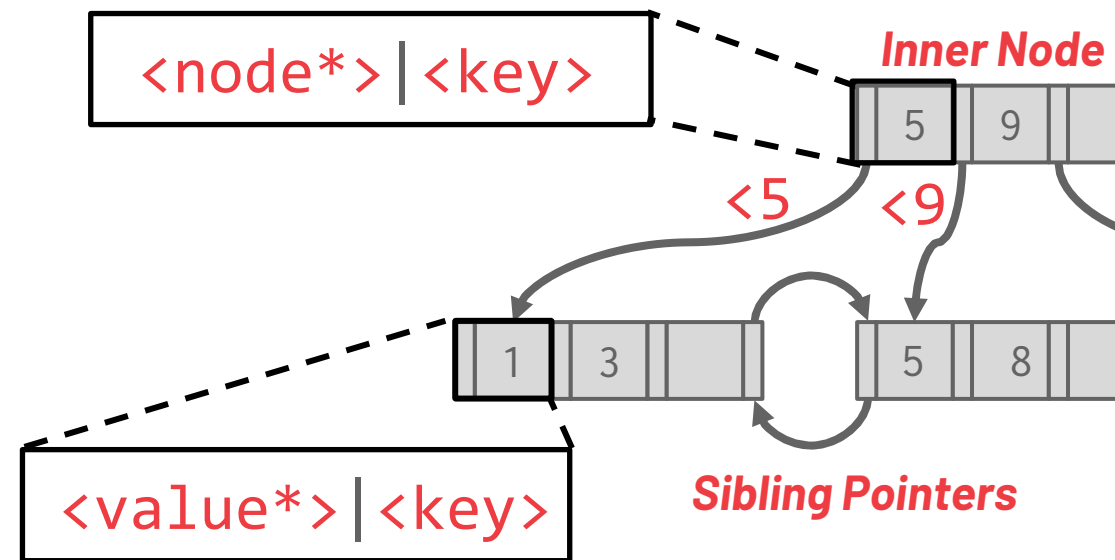
B+Tree: overview (II)

Self-balancing tree data structure: a multi-level search tree



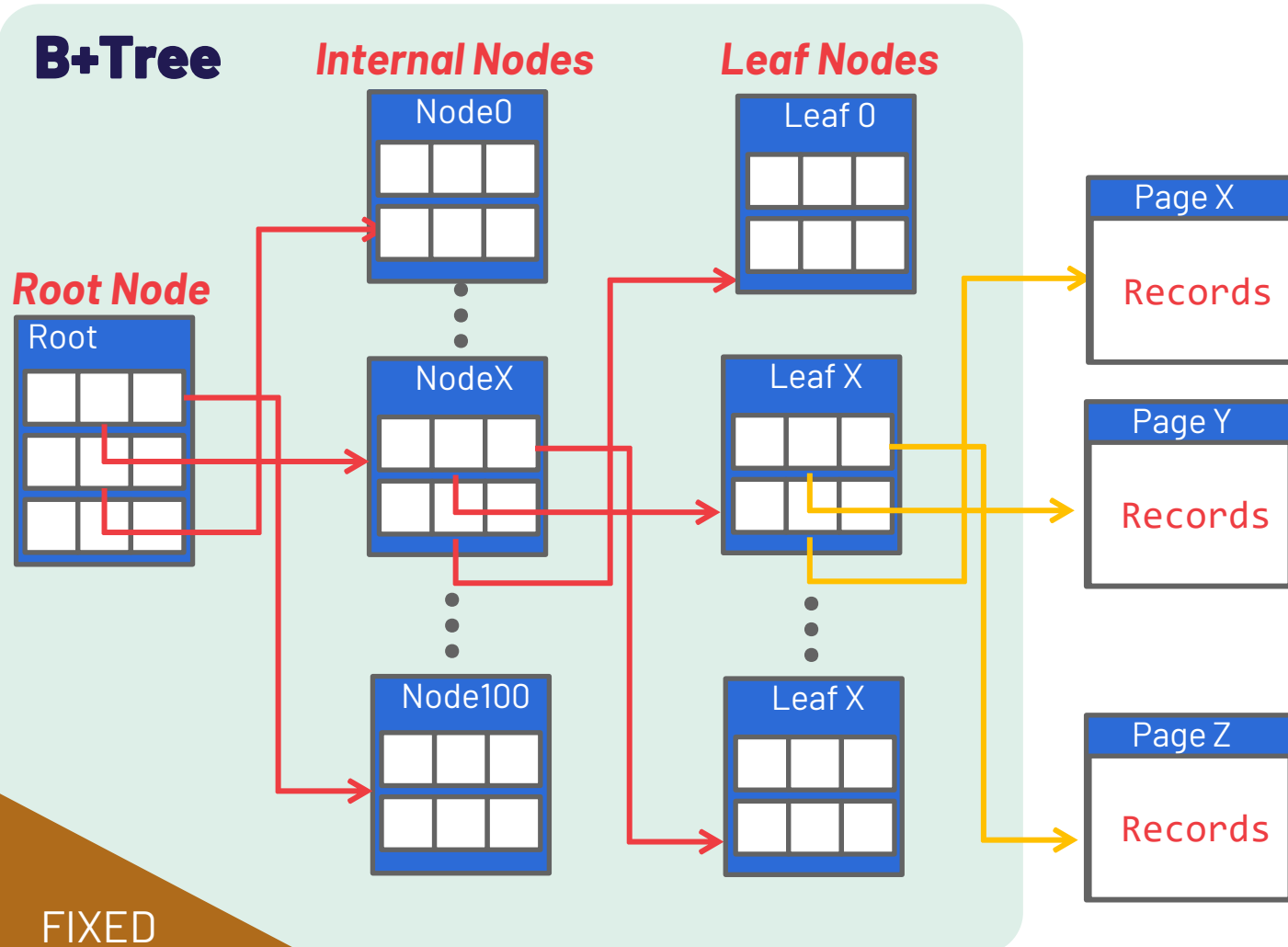
Each node is the size of a page!

The leaf node can store as values:
(a) pointer to a page of records, or
(b) the actual primary key



★ B+Tree: overview (II)

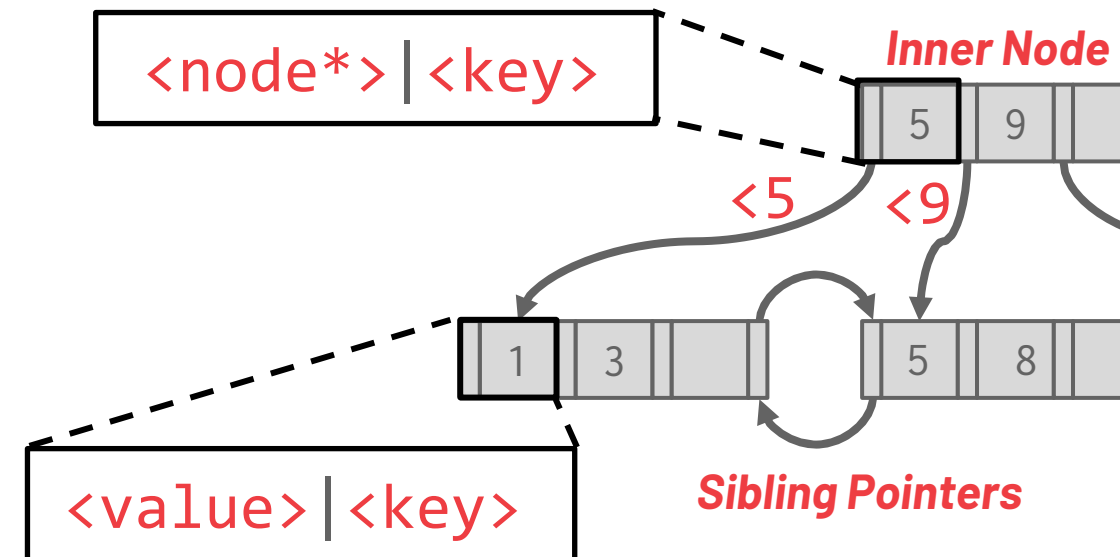
Self-balancing tree data structure: a multi-level search tree



Actual Node Size

Slower storage → larger nodes

HDD ~1MB, SSD ~10KB, in memory ~512B





B+Tree: nodes

Nodes are never full

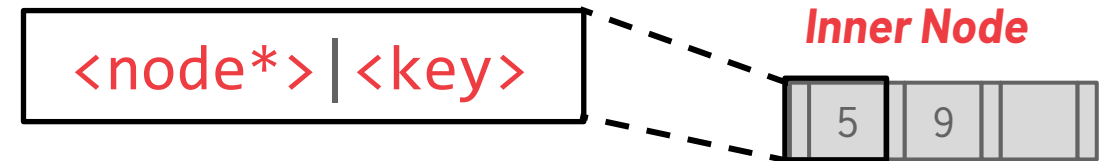
ONE B+Tree node will fit in one page

The root and the inner nodes have the same structure

With at most M-1 search keys there are **M pointers**

The pointers link to other inner nodes or leaf nodes

But not all keys are set to some values, and hence some pointers are not used



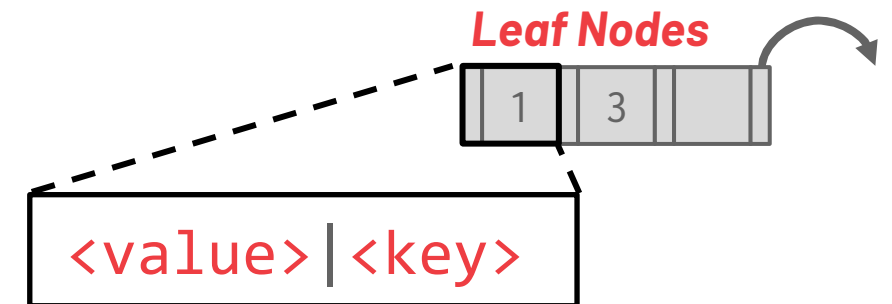
The original B-tree had values also in internal nodes
modern B+tree has only values in the leaf nodes

The leaf nodes have a different structure

The leaf nodes do not point to children nodes,
the <node*> pointer is replaced by <value*>

<value*> in leaf nodes: **can be the primary key of the record**

or the record ID itself (thus the pointer to the actual page where the record is)



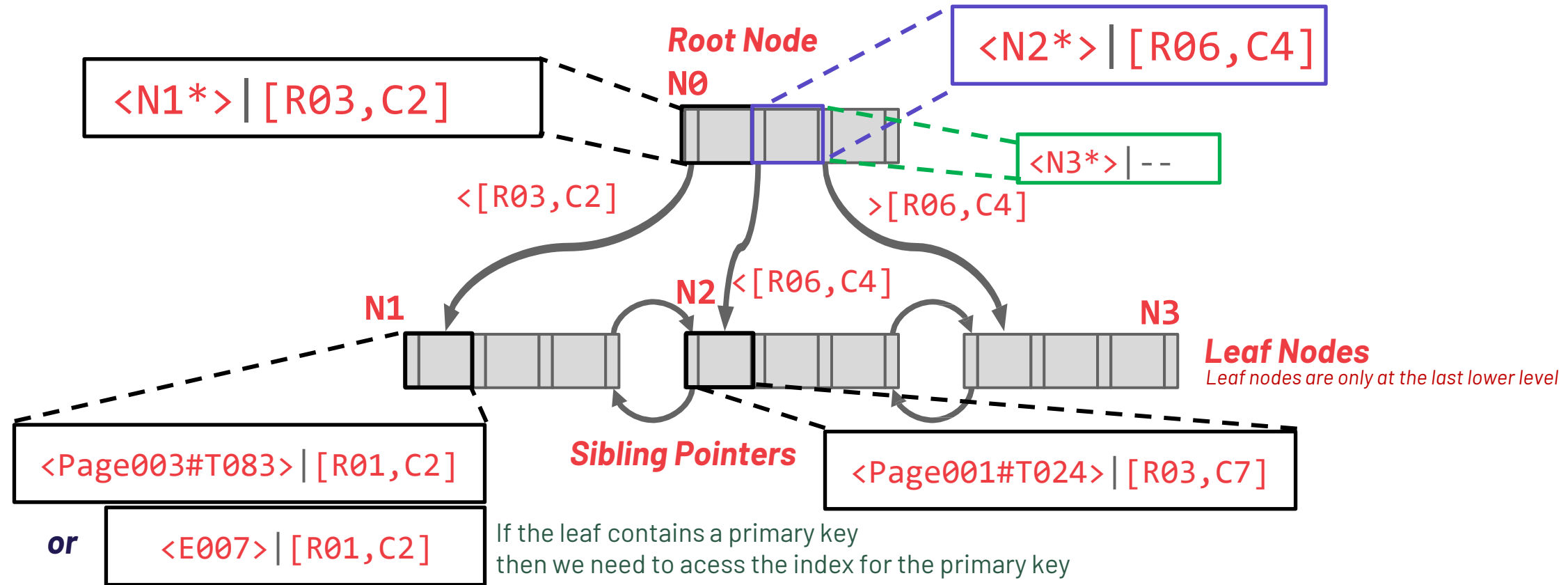
if we want to store duplicate keys, we insert the record id as part of the key also



B+Tree: Nodes

Difference between internal nodes and leaf nodes

Assume: Employee(EmployeeID, Room, Chair, Color) & B+Tree(Room,Chair)



The leaf node can store as values:
(a) pointer to a page of records, or
(b) the actual primary key

A key can be composite of multiple values
e.g., $\langle \text{profID, courseID} \rangle$

B+Tree: leaf nodes in practice

ONE B+Tree node will
fit in one page

Approach #1: Store Record IDs

Store a pointer to the location of the tuple to which the index entry corresponds.



Approach #2: Store "Tuple Data"

The leaf nodes store the actual contents of the tuple (if primary index).

Secondary indexes must store only the Record ID as their values,

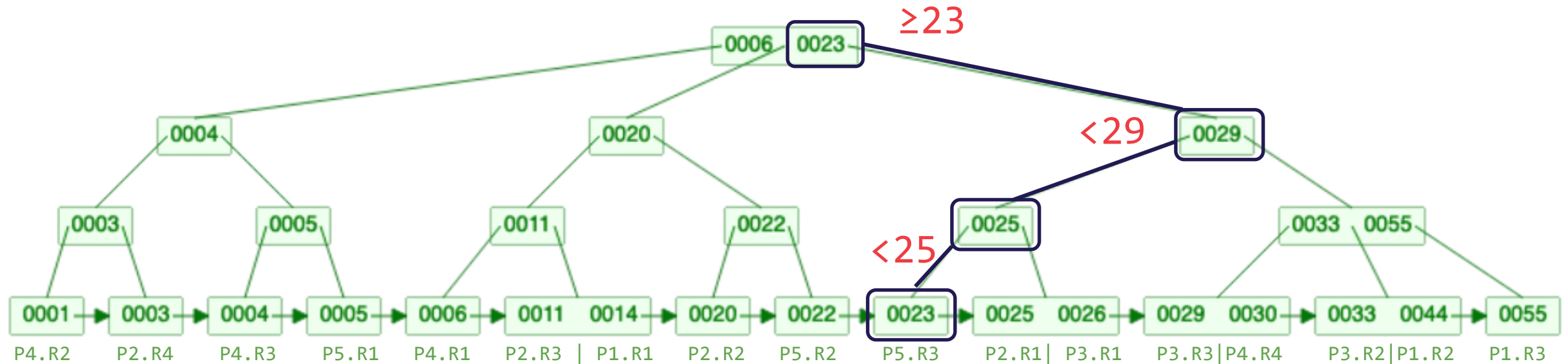
the record ID gives access directly in the primary index leaves



★ B+Tree: SEARCH (I)

B+tree supports key search in $\log(N)$ $\log(17) \simeq 4$

GET =0023 →P5.R3

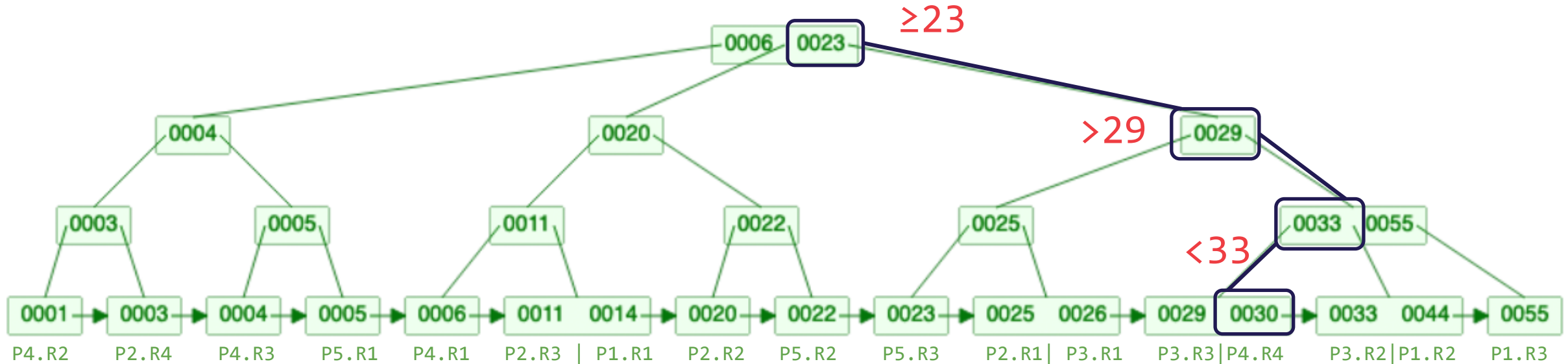


★ B+Tree: SEARCH (II)

B+tree supports key search in $\log(N)$ $\log(17) \simeq 4$

GET =0023 → P5.R3

GET =0031 → ∅



★ B+Tree: SEARCH (III)

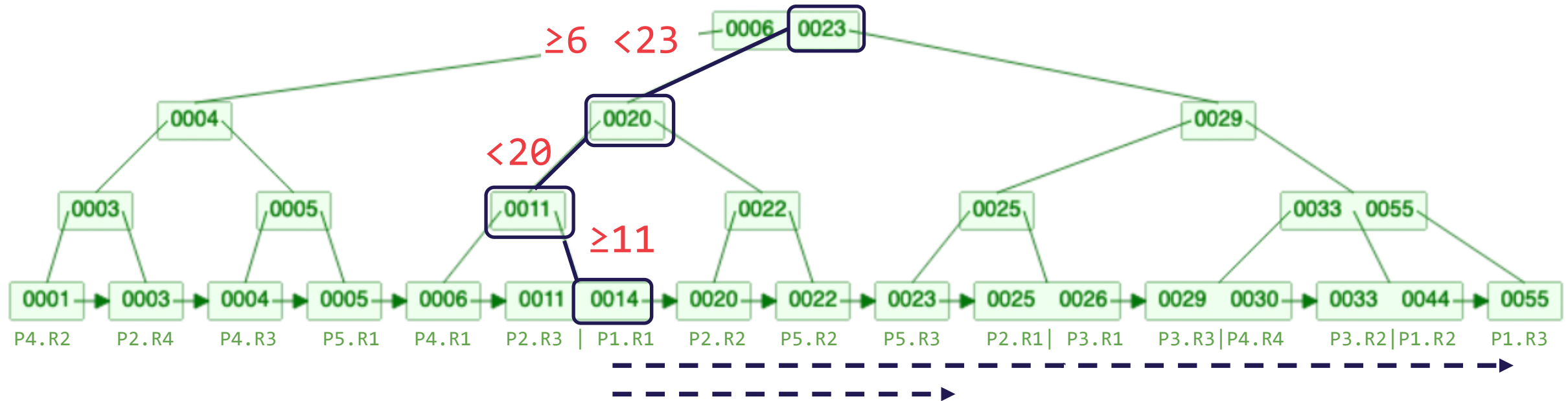
B+tree support range search in $\log(N)+N$

GET = **0023** $\rightarrow$ P5.R3

GET $\geq 14 \rightarrow$ P1.R1, P2.R2, ..., P1.R3

GET = **0031** $\rightarrow \emptyset$

GET $\geq 14 < 25 \rightarrow$ P1.R1, P2.R2, P5.R2, P5.R3



B+tree supports also Max & Min Search in $\log(N)$



B+Tree: INSERT

Nodes are never full, if full: split!

To INSERT $\langle V \rangle$ (search + insert + maybe split):

1. **Start from Root**
2. **Follow key-pointers** to the right leaf node **L**
3. **insert $\langle V \rangle$ in L** in correct position
4. **if $\text{size}(L) \leq M-1$: done!** else split

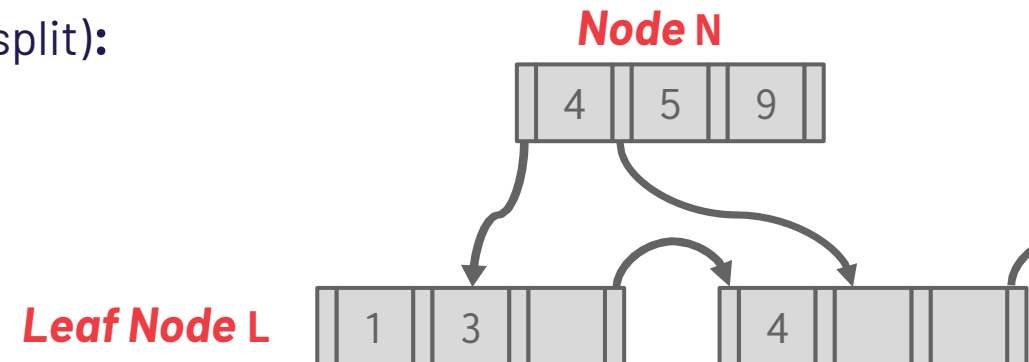
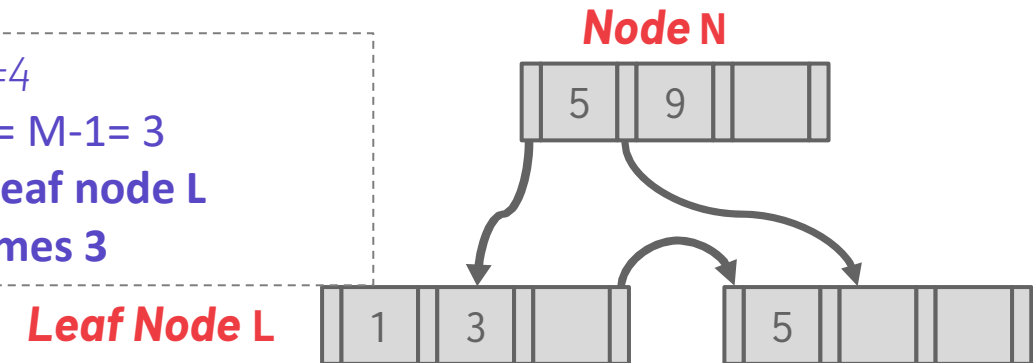
To Split leaf node **L** (new node + distribute + push up + maybe split):

1. **Create** new node L_2
2. **Distribute**: move half values to L_2
3. **Push up**: Decide new middle key $\langle V' \rangle$, Find N_p parent of **L**
Insert pointer to L_2 in N_p parent of **L**
4. **Push up can cause new split** if $\text{size}(L_p) = M-1$

This guarantees that a B+tree is never 100% filled

**Nodes have never more than $M-1$ values!
They split when they have all $M-1$ slot full**

Consider $M=4$
 $\text{max size}(N) = M-1 = 3$
insert(4) in leaf node L
size(L) becomes 3



**Splitting can “bubble up” until the root
This is how the B+Tree gets higher**



B+Tree: DELETE

Nodes are never empty, if $<M/2$: steal!

This guarantees that a B+tree is always at least 50% filled

To DELETE $\langle V \rangle$ (search + maybe delete + maybe merge):

1. **Start from Root**
2. **Follow key-pointers** to the right leaf node **L**
3. **IF $\langle V \rangle$ in L delete**
4. **if $\text{size}(L) \geq \text{ceil}(M/2) - 1$: done!** else "consolidate"*

Inner Node N

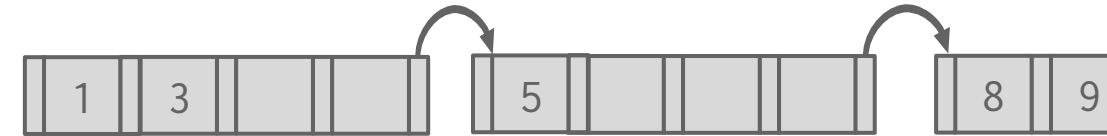


Consider $M=5$
 $\text{min-size}(N) = \text{floor}(M/2) = 2$
delete(9) from inner node N
size(N) becomes 1

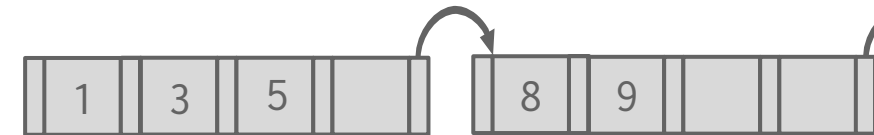
**Nodes have never less than $M/2$ values!
(only root can!)**

To Consolidate leaf node L (try borrow or try merge + update parent):

1. **Try borrow values from** sibling nodes L_{s1} **or** L_{s2}
2. **If both too small: merge**
3. **Update N_p parents of L and L_{s*}**



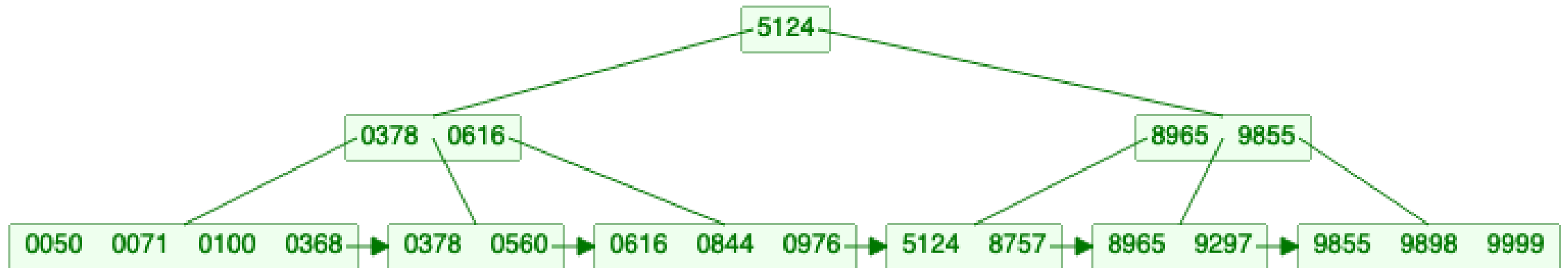
Inner Node N is undersized!



**Merging can "bubble up" until the root
This is how the B+Tree gets shorter**

Demo Time

<https://www.cs.usfca.edu/~galles/visualization/BPlusTree.html>



Only for illustrative purposes, the tool does not follow exactly all the rules!

Outline

1. Storage Architecture

- DBMS Stack ★
- Memory hierarchy ★
- Disk Oriented DBMS ★
- Sequential/Random access ★
- Page storage Architecture ★

2. Representation on Disk

- Page layout ★
- Tuple storage + tuple layout ★
- Row storage vs column storage ★
- Denormalized storage

3. Indexing

- Data Structures ★
- Assumption / requirements ★
- Types of indexes: Hash / Tree + Static / Dynamic ★
- Hash Table / Hash Index Variants ★
- Tree index ★
- Primary key and clustering ★
- Sparse / dense ★

4. When to use indexes & effect on queries

- selecting which index to build ★
- partial keys ★
- selecting which index is compatible with your query ★

★ **Core Concepts: you need to understand these**

Based on chapter & online slides:

12.1, 12.3, 12.6, 13.1-13.3, 14.1-14.3, 14.5

- from Database System Concepts 7th edition ; Silberschatz et al.

Intro to Database Systems (15-445/645), Andy Pavlo CMU

- <https://15445.courses.cs.cmu.edu/fall2022/>



Outline

1. Storage Architecture

- DBMS Stack ★
- Memory hierarchy ★
- Disk Oriented DBMS ★
- Sequential/Random access ★
- Page storage Architecture ★

2. Representation on Disk

- Page layout ★
- Tuple storage + tuple layout ★
- Row storage vs column storage ★
- Denormalized storage

3. Indexing

- Data Structures ★
- Assumption / requirements ★
- Types of indexes: Hash / Tree + Static / Dynamic ★
- Hash Table / Hash Index Variants ★
- Tree index ★
- Primary key and clustering ★
- Sparse / dense ★

4. When to use indexes & effect on queries

- selecting which index to build ★
- partial keys ★
- selecting which index is compatible with your query ★

★ **Core Concepts: you need to understand these**

Based on chapter & online slides:

12.1, 12.3, 12.6, 13.1-13.3, 14.1-14.3, 14.5

- from Database System Concepts 7th edition ; Silberschatz et al.

Intro to Database Systems (15-445/645), Andy Pavlo CMU

- <https://15445.courses.cs.cmu.edu/fall2022/>





Compatible Indexes and access patterns

Consider Relation R: $\langle A, B, C, D \rangle$

For which of the following search operations is the index guaranteed to be faster than linear scan?

Search for	Hash<A>	Hash<C,A>	B+Tree<A>	B+Tree<A,C>	B+Tree<C,A>
A=10	✓	✗	✓	✓	✗
C=10	✗	✗	✗	✗	✓
A>10	✗	✗	✓	✓	✗
C>10	✗	✗	✗	✗	✓
A=10 & C=7	?	✓	?	✓	✓
C>10 & A=7	?	?	?	✓	✓

Order Matters!

? = in theory yes, but it depends on the skew and domain of the two attributes values

The B+Tree can support prefix search for partial keys

Few systems actually support also partial prefix out of order, e.g., B=10 with index on $\langle A, B, C \rangle$



Table Indexes (II)

The **DBMS is demanded to “pick”** the best index available to execute a query efficiently
(more on this later)

There is always a tradeoff:

- more indexes → easier to run **efficient queries**
- more indexes → **more space** required on **disk**
- more indexes → **more space** occupied in **main memory**
- more indexes → **more work** to do when inserting or deleting

Deciding which index to build is not always an obvious choice...

*Do we want to speed up average queries or slowest queries?
(e.g., 90th percentile)*

take a horse they said



you will be much
faster they said

Physical Design Decisions & Tuning

Design Decisions

- Ordered/Clustering vs. Hashing ?
- Spars vs. Dense Index ?
- Clustering vs. non-clustering ?

Performance Tuning

- How often we need to “reorganize” the data structure?
- How frequent are insertions & deletions?
- How often we need range queries? Which attributes?

Not all DBMS allow to take all the decisions

Creating indexes in PostgreSQL

```
CREATE INDEX idx_ects  
ON course USING BTREE (ects);
```

```
CREATE INDEX idx_ects  
ON course USING HASH (ects);
```

- Storing tuples on disk as pages, minimize page I/O
- Fixed and Variable size records
- Dynamic vs. Static indexes
- Hashing vs. Trees
- Handling PUT/GET/DELETE
- Handling redundant keys
- Primary vs. Secondary indexes
- Indexes compatible with queries

